



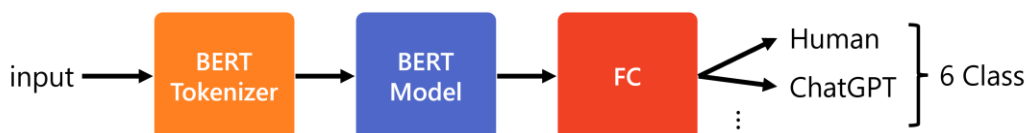
به نام خدا

گزارش پروژه نهایی درس یادگیری عمیق

نوید فرمehنی فراهانی – متین آرمان مهر – سجاد هاشم بیگی

۱. پیاده سازی مدل BERT

مدل BERT با دریافت کلمات ورودی به صورت tokenize شده، یک بردار $\mathbb{R}^{768}$ تولید می کند که یک representation از جملات ورودی خود می باشد. سپس در خروجی این مدل یک شبکه Fully Connected به منظور طبقه بندی کلاس مرتبط با هر جمله (Human، ChatGPT، ...) قرار داده می شود. در کل ۶ کلاس داریم که ۵ کلاس آن شامل متن های تولید شده توسط مدل های مولد مختلف و ۱ کلاس آن توسط انسان نوشته شده است و هدف این است که مدلی را آموزش دهیم که کلاس یک متن را تشخیص دهد. بنابراین آموزش مدل BERT به منظور طبقه بندی منبع تولید جمله به صورت زیر حاصل می شود:



شکل ۱-۱- معماری شبکه BERT Classifier

حال با توجه به معماری شبکه، مدل BERT را پیاده سازی می کنیم. ابتدا کتابخانه های لازم را وارد می کنیم:

```
!pip -q install transformers
import torch
import numpy as np
from torch.utils.data import DataLoader, Dataset
import torch.nn as nn
from transformers import AdamW
from transformers import get_linear_schedule_with_warmup # for scheduler!
import matplotlib.pyplot as plt
import torch
from transformers import AutoTokenizer, AutoModel
from transformers import BertTokenizer, BertForSequenceClassification
import random
import json
import time
```

مدل پیش آموزش داده شده BERT از روی ماژول transformers وارد می شود. در ادامه طبق مطلوب مسئله، مقدار random seed برابر عدد ۴۲ قرار داده می شود:

```
seed_val = 42
random.seed(seed_val)
np.random.seed(seed_val)
torch.manual_seed(seed_val)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed_val)
```

حال دیتاست را وارد فضای google colab می‌کنیم:

```
!gdown 1oh9c-d0fo3NtETNySmCNLUc6H1j4dSWE
!gdown 1k5LMwmYF7PF-BzYQNE2ULBae79nbM268
```

دستور gdown با وارد کردن id فایل ذخیره شده در google drive آن را دانلود می‌کند. حال در ادامه تابع to_device و DeviceDataLoader را تعریف می‌کنیم:

```
# We use this function for changing the device
def to_device(data, device):
    # for every batch, we pass the data to the current device.
    if isinstance(data, (list,tuple)):
        return [to_device(x, device) for x in data]
    return data.to(device, non_blocking=True)

# We use this class for passing the data and model to the device
class DeviceDataLoader():
    def __init__(self, dl, device):
        self.dl=dl
        self.device=device
    def __iter__(self):
        #For every batch, we pass it to the current device.
        for b in self.dl:
            yield to_device(b, self.device)
    def __len__(self):
        return len(self.dl)
```

این دو تابع درواقع مدل و دیتاست را وارد GPU می‌کند. مدل توسط تابع to_device وارد GPU می‌شود و کلاس DeviceDataLoader با دریافت DataLoader از دیتاست با دریافت هر batch، آن را به GPU می‌دهد. لذا دیگر نیازی به وارد کردن دستور to(device) در هر بیج نداریم و کافیه یکبار DataLoader به کلاس DeviceDataLoader داده شود. این کار از طرفی باعث صرفه‌جویی در استفاده از حافظه GPU خواهد شد. در ادامه متغیر device که می‌تواند cuda یا cpu باشد را مشخص می‌کنیم:

```
# Selecting the current device
if torch.cuda.is_available():
    device=torch.device('cuda')
else:
```

```
device=torch.device('cpu')
print("The device is:",device)
```

که با اجرای آن در google colab متغیر device برابر cuda می‌شود:

```
The device is: cuda
```

در ادامه تابع tokenizer به صورت زیر تعریف می‌شود:

```
bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

سپس از فایل "subtaskB_train.jsonl" دیتاست را خوانده و ستون‌های text و label را از آن جدا می‌کنیم و در یک آرایه ذخیره می‌کنیم. همین فرآیند را برای فایل "subtaskB_dev.jsonl" نیز تکرار می‌کنیم تا داده تست ذخیره شود:

```
## Load dataset and make list of texts and their labels
all_texts=[]
all_labels=[]
all_names=[]
with open('subtaskB_train.jsonl','r') as f:
    for line in f:
        data = json.loads(line)
        all_texts.append(data['text'])
        all_labels.append(data['label'])
all_texts_test=[]
all_labels_test=[]
all_names_test=[]
with open('subtaskB_dev.jsonl','r') as f:
    for line in f:
        data = json.loads(line)
        all_texts_test.append(data['text'])
        all_labels_test.append(data['label'])
```

حال کلاس دیتاست را تعریف می‌کنیم:

```
class BERT_Dataset(Dataset):
    def __init__(self, text, labels,label_masks, max_len):
        self.text = text
        self.labels = np.array(labels)# np.array!!
        self.label_masks=label_masks
        self.max_len = max_len
    def __getitem__(self,index):
        tokenized_text = bert_tokenizer.encode_plus(self.text[index],
                                                    max_length = self.max_len,
                                                    return_tensors="pt",
                                                    truncation = True,
```

```

        add_special_tokens=True,
        padding="max_length",
        return_attention_mask=True
    )

    return tokenized_text.input_ids.squeeze(), tokenized_text.attention_mask.squeeze(),
    tokenized_text.token_type_ids.squeeze(),self.labels[index],self.label_masks[index]
def __len__(self):
    return len(self.text)

```

عمل توکن کردن متن‌های ورودی در تابع `__getitem__` انجام می‌شود و در خروجی آن تانسور توکن شده متن، ماسک توجه (به منظور نادیده گرفتن توکن‌های padding)، نوع توکن‌ها، کلاس داده‌ها و ماسک کلاس داده‌ها برگردانده می‌شود. ماسک کلاس داده‌ها برای داده‌هایی که بدون کلاس هستند برابر False یا صفر تعریف می‌شوند.

در ادامه آرایه داده‌ها را با ترتیب تصادفی تغییر می‌دهیم. زیرا داده‌ها به ترتیب کلاس‌ها قرار دارند و در صورتی که بخواهیم از درصدهای مشخصی از داده‌های با برچسب برای آموزش شبکه BERT استفاده کنیم، از کلاس‌های مختلف به طور مساوی داده آموزشی در اختیار داشته باشیم:

```

the_list=list(range(0,len(all_texts)))
selector=the_list[0:int(len(all_texts))]
np.random.shuffle(selector)
all_texts_shuffled=[]
all_labels_shuffled=[]
for i in range(len(selector)):
    all_texts_shuffled.append(all_texts[selector[i]])
    all_labels_shuffled.append(all_labels[selector[i]])

```

حال هایپرپارامترهای مسئله را تعریف می‌کنیم:

```

learning_rate = 2e-5
epochs = 3
percentage=[0.01,0.05,0.1,0.5]
batch_size = 128
max_len=64

```

با توجه به اینکه در مدل BERT محدودیت بُعد تانسور ورودی (حداکثر ۵۱۲ بُعد) وجود دارد. مجبور هستیم که طول متن‌های ورودی به مدل را به اندازه متغیر `max_len` محدود کنیم. ما برای تسریع سرعت یادگیری شبکه و محدودیت حافظه GPU که در اختیارمان هست، از جملات با حداکثر ۶۴ توکن استفاده می‌کنیم.

در انتهای این بخش می‌خواهیم مدل را آموزش دهیم. ابتدا دیتاست و `DataLoader` های مربوط به داده‌های آموزشی و تست را تعریف کرده و نوع `optimizer` و بردارهای مربوط به `accuracy` مدل را مقدار دهی اولیه می‌کنیم. در انتها هر یک از `Accuracy`

های محاسبه شده در بخش Test برای درصدهای مختلف داده‌های لیبلدار را در متغیر `average_accs_for_all_percentages` ذخیره می‌کنیم.

```
average_accs_for_all_percentages=[]

test_dataset = BERT_Dataset(text = all_texts_test,
                             labels = all_labels_test,
                             label_masks=torch.ones(len(all_labels_test)).to(device),
                             max_len = max_len)
test_loader = DataLoader(test_dataset, batch_size = batch_size, shuffle = True)
test_loader = DeviceDataLoader(test_loader, device)
loss_fn = torch.nn.CrossEntropyLoss()
```

در انتها توسط حلقه `for` به ازای درصد مشخصی از داده‌های برچسب دار در دیتاست، هربار مدل را آموزش داده و دقت طبقه بندی را در داده تست ذخیره می‌کنیم. در این بخش از داده‌های بدون برچسب برای آموزش شبکه استفاده نشده است و فقط از درصدهای مشخص داده‌های برچسب دار استفاده شده است. در شبکه `BertForSequenceClassification` در انتهای لایه آخر با 768 فیچر، یک لایه FC قرار داده شده تا طبقه بندی را انجام دهد. لازم به ذکر است که در ابتدا، از تابع هزینه `loss_fn = torch.nn.CrossEntropyLoss()` استفاده شده بود؛ اما چون تفاوت چندانی با `loss` شبکه BERT نداشت، این دستور کامنت شد. لذا در این مورد ما `loss` شبکه را از خروجی مدل گرفتیم که یک نوع `binary cross entropy` را در خود دربر دارد. در انتها نمودار دقت نهایی عملکرد طبقه‌بند شبکه در دیتای دست به ازای درصدهای مختلفی که از دیتای برچسب دار استفاده کردیم، قابل نمایش است:

```
for percentage_index in range(len(percentage)):
    print("#####\n\n")
    print("This model considers",100*percentage[percentage_index],"% of the data for supervised learning,
so all other data are redundant in part 1")
    # We Define a mask for implementing semi-supervised, we won't use this mask in part 1.
    all_labels_masks=torch.ones(len(all_labels),dtype=bool).to(device)
    for i in range(int(percentage[percentage_index]*len(all_texts_shuffled)),len(all_texts_shuffled)):
        all_labels_masks[i]=False
    train_dataset_with_label = BERT_Dataset(text =
all_texts_shuffled[:int(percentage[percentage_index]*len(all_texts_shuffled))],
        labels =
all_labels_shuffled[:int(percentage[percentage_index]*len(all_texts_shuffled))],
        label_masks=all_labels_masks[:int(percentage[percentage_index]*len(all_text
ts_shuffled))],

        max_len = max_len)
    train_loader_with_label = DataLoader(train_dataset_with_label, batch_size = batch_size, shuffle =
True)
    train_loader_with_label = DeviceDataLoader(train_loader_with_label, device)

    print("We have ",len(train_dataset_with_label),"Data for train")
```

```

print("We have ",len(test_dataset),"Data for test")
total_steps = len(train_loader_with_label) * epochs

# We have 6 classes for classification, so we have:
num_class = 6
bert_model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=num_class)
bert_model = to_device(bert_model, device)
optimizer = torch.optim.AdamW(bert_model.parameters(), lr=learning_rate)
    scheduler = get_linear_schedule_with_warmup(
        optimizer,
        num_warmup_steps=0,
        num_training_steps=total_steps
    )

total_loss_for_all_epochs=[]
total_acc_for_all_epochs=[]
print("\n\n\n\nTraining has been started!!")
print("##### \n\n\n\n\n")
tic=time.time()
for epoch in range(epochs):
    bert_model.train()
    total_loss=[]
    total_acc=[]
    total_samples= 0
    total_correct= 0
    i=0
    for batch in train_loader_with_label:
        i=i+1
        input_ids, attention_mask, token_type_ids, label,_ = batch

        optimizer.zero_grad()

        outputs = bert_model(input_ids, attention_mask=attention_mask, token_type_ids=token_type_ids,
labels=label)
        #loss=loss_fn(outputs.logits,label)
        loss = outputs.loss
        total_loss.append(loss.item())
        total_loss_for_all_epochs.append(loss.item())
        loss.backward()
        optimizer.step()
        scheduler.step()
        print("Step:",i," From",len(train_dataset_with_label)//batch_size, " Loss:",loss.item())

```

لازم به ذکر است که برای عملکرد سریع تر

در کد زیر، به منظور تصمیم‌گیری برای لیبل خروجی مدل، بیشترین مقدار برای خروجی مدل که به صورت one-hot است را در نظر می‌گیریم که اندیس آن، نشانگر لیبل پیش‌بینی شده است (چون لیبلها از ۰ تا ۵ هستند، اندیسها همان لیبلها خواهند بود) و درصدی از آنها که با لیبلها برابر هستند، به عنوان Accuracy برای Train اعلام می‌گردد. و نتیجه هر ۵ بار اجرا، به کاربر نشان داده می‌شود.

```
_, preds = torch.max(outputs[1], dim = 1)
total_samples += label.size(0)
total_correct += (preds == label).sum().item()

if i%5 == 0:
    acc_train=100*total_correct/total_samples

    print("Train Accuracy For 20 Steps: ",acc_train,"%")
    total_correct=0
    total_samples=0
    total_acc.append(acc_train)
    total_acc_for_all_epochs.append(acc_train)

    print(f"Epoch {epoch + 1}/{epochs}")
    print("Average Train Loss:", torch.mean(torch.tensor(total_loss)), "Average Train Accuracy:",
torch.mean(torch.tensor(total_acc)))

toc=time.time()
print("Elapsed time is",round(toc-tic,5),"seconds")
plt.plot(total_loss_for_all_epochs)
plt.title("All losses for all epochs")
plt.ylabel("Train Loss")
plt.xlabel("steps")
plt.show()
plt.plot(total_acc_for_all_epochs)
plt.title("All accuracies for all epochs")
plt.ylabel("Train Accuracy")
plt.xlabel("steps")
plt.show()
print(len(train_dataset_with_label))

bert_model.eval()
total_loss = 0.0
i = 0
```

```

total_loss=[]
total_acc=[]
total_samples= 0
total_correct= 0
for batch in test_loader:
    i += 1
    input_ids, attention_mask, token_type_ids, label, label_mask = batch

    outputs = bert_model(input_ids, attention_mask=attention_mask, token_type_ids=token_type_ids,
labels=label)
    loss = outputs.loss
    total_loss.append(loss.item())
    _, preds = torch.max(outputs[1], dim = 1)
    total_samples += label.size(0)
    total_correct += (preds == label).sum().item()
    if i%5 == 0:
        acc=100*total_correct/total_samples
        total_acc.append(acc)
        print("Step:",i, "From",len(test_dataset)//batch_size)
        print('Tes loss:',loss.item(), 'Preds:',preds,"Labels:",label)
        print("Test Accuracy For 20 Steps: ",acc,"%")
        total_correct=0
        total_samples=0
average_loss =torch.mean(torch.tensor(total_loss))
average_acc =torch.mean(torch.tensor(total_acc))
print("Average Test Loss:",average_loss,"\nAverage Test Accuracy:",average_acc)
average_accs_for_all_percentages.append(average_acc)

```

با اجرای کد بالا مدل به حالت یادگیری رفته و در درصدهای مختلفی از دیتاست لیبل دار که تعیین کردیم دقت را روی داده آموزشی و تست، و مقدار loss شبکه در زمان آموزش اندازه گیری کرده و ذخیره می کند. بنابراین می توانیم نمودار آن را رسم کنیم و اطلاعات آن را در یک فایل xlsx ذخیره کنیم:

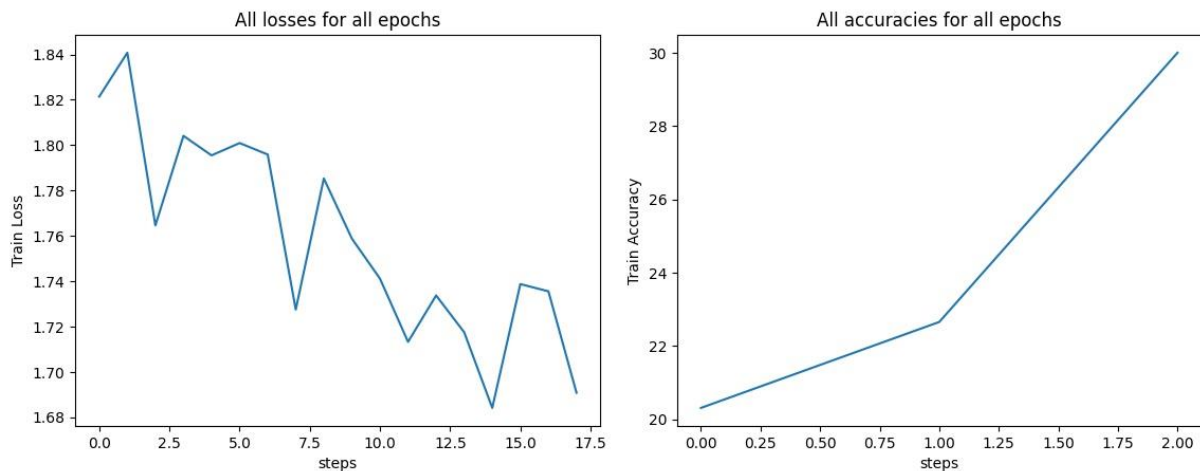
```

plt.plot(percentage,average_accs_for_all_percentages, marker='o', linestyle='--')
plt.title("Average Test Accuracy for different Percentages")
plt.ylabel("Test Accuracy")
plt.xlabel("percentage")
plt.show()
df = pd.DataFrame({'percent': percentage, 'acc': average_accs_for_all_percentages})

# Save the DataFrame to an Excel file
df.to_excel('data.xlsx', index=False)
print("Data saved to data.xlsx")

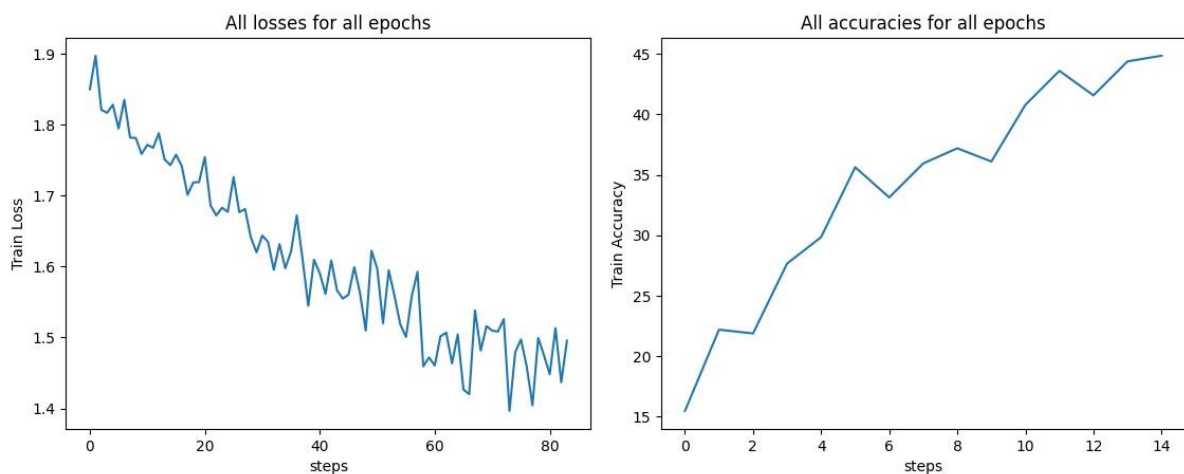
```

دقت و loss آموزش مدل در استفاده از ۱٪ داده های آموزشی لیبل دار به صورت زیر است:



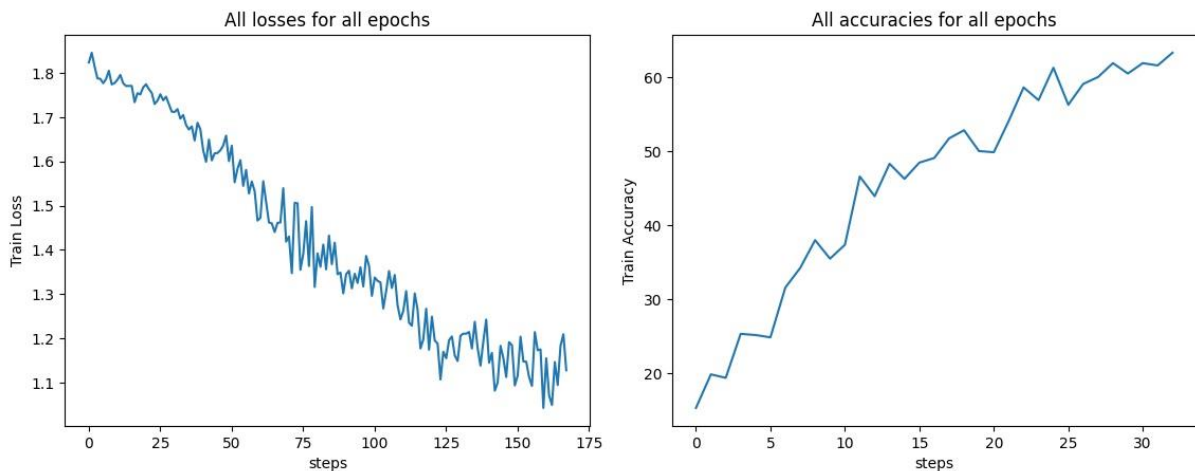
شکل ۱-۲- دقت و loss مربوط به مدل BERT توسط آموزش با ۱٪ داده‌های آموزشی

دقت و loss آموزش مدل در استفاده از ۵٪ داده‌های آموزشی لیبیل دار به صورت زیر است:



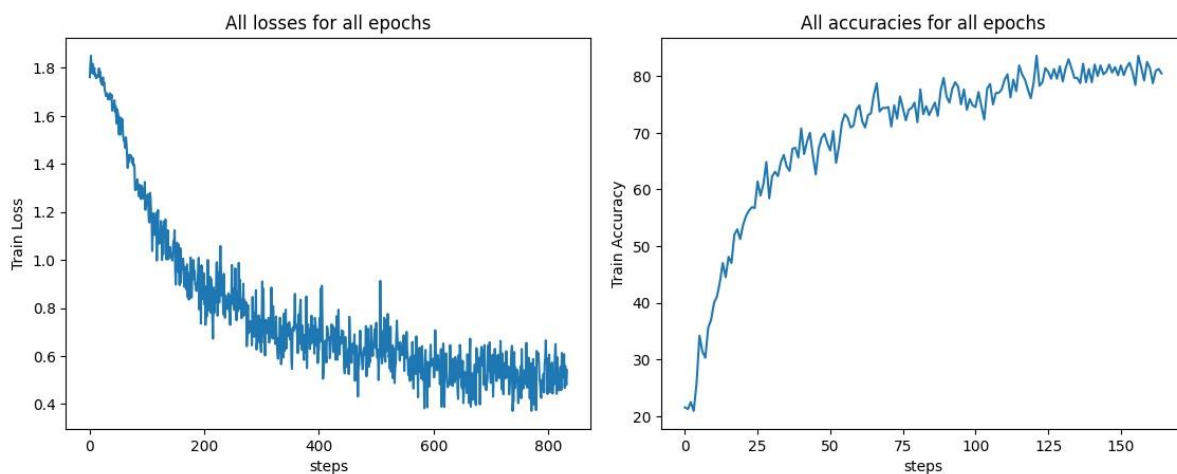
شکل ۱-۳- دقت و loss مربوط به مدل BERT توسط آموزش با ۵٪ داده‌های آموزشی

دقت و loss آموزش مدل در استفاده از ۱۰٪ داده‌های آموزشی لیبیل دار به صورت زیر است:



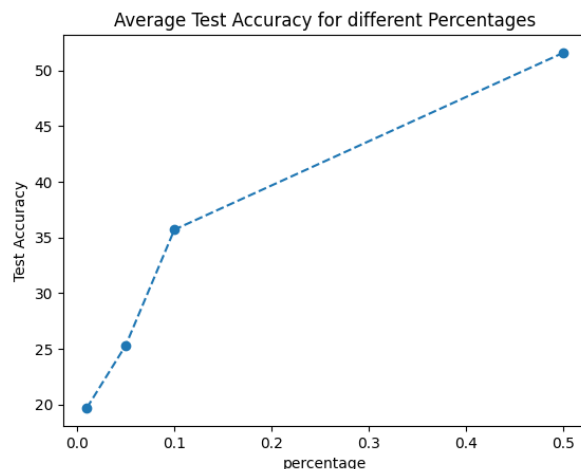
شکل ۴-۱- دقت و loss مربوط به مدل BERT توسط آموزش با ۱۰٪ داده‌های آموزشی

دقت و loss آموزش مدل در استفاده از ۵۰٪ داده‌های آموزشی لیبیل دار به صورت زیر است:



شکل ۵-۱- دقت و loss مربوط به مدل BERT توسط آموزش با ۵۰٪ داده‌های آموزشی

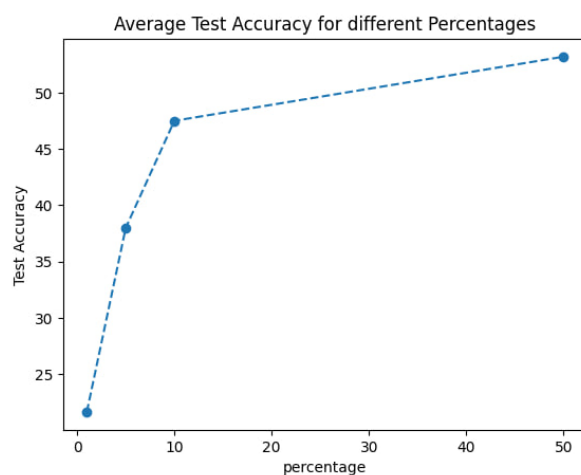
در نهایت میانگین دقت مدل روی دیتاست تست به ازای درصدهای مختلفی که شبکه از داده‌های لیبیل دار آموزش داده شده است، به صورت زیر حاصل می‌شود:



شکل ۱-۶- نمودار دقت مدل روی داده تست به ازای استفاده از درصدهای مشخصی از داده‌های برچسب دار به ازای Max_length=64

با توجه به نتایج حاصل طبیعی است که با افزایش درصد استفاده از داده‌های برچسب دار، دقت مدل افزایش یابد زیرا از generalization بالاتری برخوردار هستیم. در این بخش از بقیه داده‌های بدون برچسب استفاده نشده است بنابراین مشاهده می‌شود که مدل خیلی دقت خوبی ندارد.

به منظور دستیابی به نتیجه بهتر، می‌توان Hyper Parameter های مسئله، از جمله Max_length برای فرآیند تبدیل به Token، و همچنین تعداد Epoch ها را افزایش داد که ما، در اینبار، Max_lenght را ۱۲۸ در نظر گرفتیم و نتیجه زیر حاصل شده است.



شکل ۱-۷- نمودار دقت مدل روی داده تست به ازای استفاده از درصدهای مشخصی از داده‌های برچسب دار به ازای Max_length=128

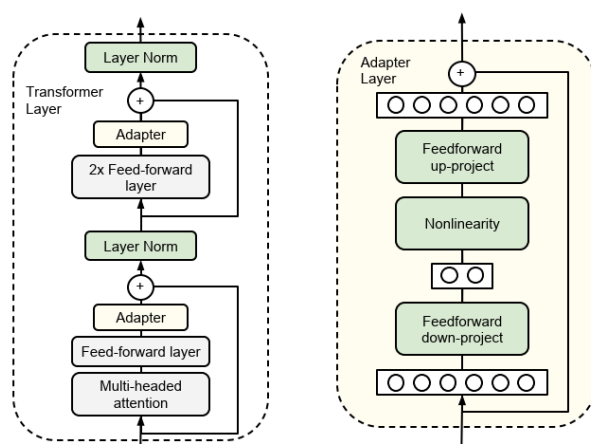
۲. استفاده از بلوک Adapter

در قسمت اول، از یک مدل از قبل آموزش داده شده استفاده شده بود و لذا به طور کامل Fine Tune میشد؛ به عبارت دیگر، مدل با شرایط اولیه ی مدل بارگیری شده آموزش داده میشد. اما این کار باعث میشد که تمام پارامترهای آن learn شوند و لذا آموزش شبکه، کند می گردد.

حال، در این قسمت از Adapter ها استفاده میکنیم که قادر هستند تا فقط یک ماژولی را به شبکه اضافه کنند و طبق مقاله [1] فقط این ماژول، و ماژولهای nn.LayerNorm آموزش داده میشوند؛ لذا تمام ماژولهای دیگر Freeze خواهند ماند. بنابراین باید معماری طراحی شود که قادر باشد، گذر گرادیان از ماژولها را تنظیم کند.

در این بخش، ۲ رویکرد برای پیاده سازی Adapter پیشنهاد شده است؛ رویکرد اول، نوشتن کلاس Adapter و قرار دادن آن، در مدل Bert، با در نظر گرفتن چگونگی تنظیم گرادیانها است. رویکرد دوم، استفاده از کتابخانه ها و توابع آماده است تا مدل بتواند یک بلوک Adapter را در خود داشته باشد تا فقط گرادیان آن و گرادیان nn.LayerNorm قابل یادگیری باشند.

شکل Adapter به صورت شکل ۱-۲ است [1] که فقط کافیسیت تا یک شبکه MLP ساده ۲ لایه را در نظر بگیریم که دارای یک عملگر غیر خطی است که برای این عملگر، از تابع Tanh استفاده شد؛ همچنین این مدل دارای یک Skip Connection است.



شکل ۱-۲ - بلوکهای Adapter و یک لایه ی Transformer - شکل سمت راست مربوط به Adapter و شکل سمت چپ مربوط به Transformer هستند.

روش اول: پیاده سازی Adapter به صورت From Scratch

در کلاس Adapter طراحی شده فقط کافیسیت تا از آخرین لایه خطی ماژول قبل به یک بعد کمتر که به صورت m در نظر گرفته شده است برویم؛ این عدد باید بسیار کمتر از آخرین لایه خطی ماژول قبلی باشد (که ۷۶۸ است) که برای آن، یک عدد کوچکتر از ۷۶۸ (به عنوان مثال ۵۱۲) لحاظ شد؛ در صورتی که این عدد زیاد باشد، مدل میتواند بهتر آموزش داده شود اما زمان بیشتری برای شبیه سازی نیاز خواهد بود؛ زیرا ماژولهای Trainable بزرگتر میشوند و در صورتی که این عدد کوچک باشد، سرعت افزایش

می یابد اما دقت کاهیش می یابد ضمناً یک skip connection از ورودی به خروجی وجود دارد و Activation function غیر خطی به صورت tanh در نظر گرفته شد:

```
class AdapterForTransformer(nn.Module):
    def __init__(self, size, m):
        super(AdapterForTransformer, self).__init__()
        self.adapter_module = nn.Sequential(
            nn.Linear(size, m),
            nn.Tanh(),
            nn.Linear(m, size)
        )
        self.layer_norm = nn.LayerNorm(size, eps=1e-12)

    def forward(self, x):
        # Apply adapter module with residual connection and layer norm
        return self.layer_norm(self.adapter_module(x) + x)
```

در ادامه کلاس زیر که مدل را به یک مدل، همراه با Adapter تبدیل میکند و گرادینت های لازم را تنظیم میکند، نوشته شده است. در کلاس AdapterLayerNorm ابتدا پارامترهای لازم برای هر ماژول Adapter در نظر گرفته میشوند و طبق def forward مدل میتواند از این Adapter، آموزش داده شود.

```
class BertWithAdapters(BertForSequenceClassification):
    def __init__(self, config, m=2):
        super().__init__(config)

        # Custom layernorm module to be used in BertLayer
        class AdapterLayerNorm(nn.Module):
            def __init__(self, norm_layer):
                super().__init__()
                self.adapter = AdapterForTransformer(norm_layer.normalized_shape[0], m)

            def forward(self, x):
                return self.adapter(x)

        # Recursively traverse all layers and replace layernorm with adapter layernorm
        def replace_layernorm_with_adapters(model):
            for name, child in model.named_children():
                if isinstance(child, nn.LayerNorm):
                    setattr(model, name, AdapterLayerNorm(child))
                else:
                    replace_layernorm_with_adapters(child)

        # Replace layernorms for the BertModel part of the sequence classifier
        replace_layernorm_with_adapters(self.bert)
```

```
# Freeze all parameters in the model
self.freeze_parameters()
```

حال، با توجه به اینکه مدل Transformer دارای ۱۲ لایه موازی است، یک متد بازگشتی به صورت `replace_layernorm_with_adapters` در نظر گرفته شد تا با استفاده از یک حلقه که برای تمام ماژولها برای `model.named_children` است، ماژولها بررسی شوند تا هنگامی که به `nn.LayerNorm` رسیدیم، ماژول `Adapter` طراحی شده قرار داده شود. مجدد این نکته لازم به ذکر است که از تابع استفاده شده به صورت بازگشتی استفاده میکنیم که اگر به ماژول `nn.LayerNorm` نرسیدیم، در تمام ۱۲ لایه Bert، جستجو انجام شود تا نهایتاً ماژول `nn.LayerNorm` در هر ۱۲ لایه یافت شود. سپس در خط `replace_layernorm_with_adapters(self.bert)` تابع توضیح داده شده فراخوانی میشود و پس از آن، گرادیانها تنظیم میشوند که تابع آن به صورت زیر است:

```
def freeze_parameters(self):
    # Freeze all parameters first
    for param in self.parameters():
        param.requires_grad = False

    # Unfreeze parameters in AdapterForTransformer
    for name, sub_module in self.named_modules():
        if isinstance(sub_module, AdapterForTransformer):
            for param in sub_module.parameters():
                param.requires_grad = True
```

لذا در داخل کلاس قبلی، از متد فوق استفاده شد که در ابتدا، تمام گرادیانها `False` میشوند تا شبکه کاملاً Freeze شود و سپس در صورتی که به ماژول `Adapter` برسیم، تمام بخشهای این `sub module`، گرادیان `True` خواهند گرفت و در نهایت، با استفاده از این کد، از تمام کلاسهای نوشته شده استفاده شد، تا مدل Bert، به یک مدل Bert همراه با `Adapter` تبدیل شود که یکبار ماژولهای مختلف مدل قبل از اعمال `Adapter`، و یک بار هم بعد از اعمال `Adapter` نمایش داده شده اند؛ و سپس، نحوه گرادیانها برای یکی از لایه ها نشان داده شده است؛ لازم به ذکر است که این گرادیانها برای هر ۱۲ لایه که به صورت موازی هستند، برقرار است.

```

<bound method Module.parameters of BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=6, bias=True)
)
>>

```

شکل ۲-۲- ساختار مدل Bert، قبل از اعمال Adapter

```

(bert): BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): AdapterLayerNorm(
      (adapter): AdapterForTransformer(
        (adapter_module): Sequential(
          (0): Linear(in_features=768, out_features=64, bias=True)
          (1): Tanh()
          (2): Linear(in_features=64, out_features=768, bias=True)
        )
        (layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      )
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=768, out_features=768, bias=True)
            (LayerNorm): AdapterLayerNorm(
              (adapter): AdapterForTransformer(
                (adapter_module): Sequential(
                  (0): Linear(in_features=768, out_features=64, bias=True)
                  (1): Tanh()
                  (2): Linear(in_features=64, out_features=768, bias=True)
                )
                (layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              )
            )
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=768, out_features=3072, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=3072, out_features=768, bias=True)
          (LayerNorm): AdapterLayerNorm(
            (adapter): AdapterForTransformer(
              (adapter_module): Sequential(
                (0): Linear(in_features=768, out_features=64, bias=True)
                (1): Tanh()
                (2): Linear(in_features=64, out_features=768, bias=True)
              )
              (layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            )
          )
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (Intermediate): BertIntermediate(
      (dense): Linear(in_features=768, out_features=3072, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output): BertOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)
      (LayerNorm): AdapterLayerNorm(
        (adapter): AdapterForTransformer(
          (adapter_module): Sequential(
            (0): Linear(in_features=768, out_features=64, bias=True)
            (1): Tanh()
            (2): Linear(in_features=64, out_features=768, bias=True)
          )
          (layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=6, bias=True)
)
>>

```

شکل ۲-۳- ساختار مدل Bert، بعد از اعمال Adapter

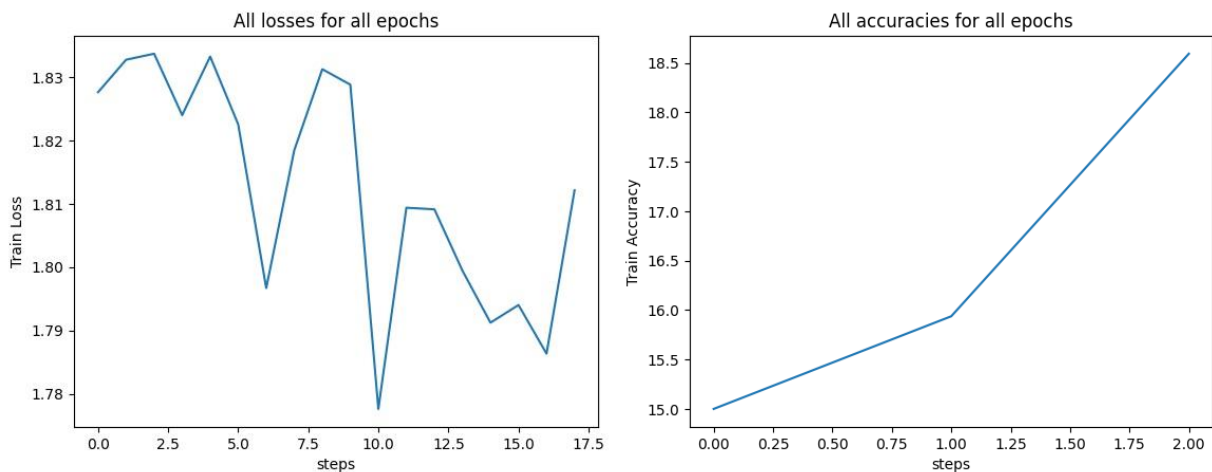
```

bert.embeddings.word_embeddings.weight False
bert.embeddings.position_embeddings.weight False
bert.embeddings.token_type_embeddings.weight False
bert.embeddings.LayerNorm.adapter.adapter_module.0.weight True
bert.embeddings.LayerNorm.adapter.adapter_module.0.bias True
bert.embeddings.LayerNorm.adapter.adapter_module.2.weight True
bert.embeddings.LayerNorm.adapter.adapter_module.2.bias True
bert.embeddings.LayerNorm.adapter.layer_norm.weight True
bert.embeddings.LayerNorm.adapter.layer_norm.bias True
bert.encoder.layer.0.attention.self.query.weight False
bert.encoder.layer.0.attention.self.query.bias False
bert.encoder.layer.0.attention.self.key.weight False
bert.encoder.layer.0.attention.self.key.bias False
bert.encoder.layer.0.attention.self.value.weight False
bert.encoder.layer.0.attention.self.value.bias False
bert.encoder.layer.0.attention.output.dense.weight False
bert.encoder.layer.0.attention.output.dense.bias False
bert.encoder.layer.0.attention.output.LayerNorm.adapter.adapter_module.0.weight True
bert.encoder.layer.0.attention.output.LayerNorm.adapter.adapter_module.0.bias True
bert.encoder.layer.0.attention.output.LayerNorm.adapter.adapter_module.2.weight True
bert.encoder.layer.0.attention.output.LayerNorm.adapter.adapter_module.2.bias True
bert.encoder.layer.0.attention.output.LayerNorm.adapter.layer_norm.weight True
bert.encoder.layer.0.attention.output.LayerNorm.adapter.layer_norm.bias True
bert.encoder.layer.0.intermediate.dense.weight False
bert.encoder.layer.0.intermediate.dense.bias False
bert.encoder.layer.0.output.dense.weight False
bert.encoder.layer.0.output.dense.bias False
bert.encoder.layer.0.output.LayerNorm.adapter.adapter_module.0.weight True
bert.encoder.layer.0.output.LayerNorm.adapter.adapter_module.0.bias True
bert.encoder.layer.0.output.LayerNorm.adapter.adapter_module.2.weight True
bert.encoder.layer.0.output.LayerNorm.adapter.adapter_module.2.bias True
bert.encoder.layer.0.output.LayerNorm.adapter.layer_norm.weight True
bert.encoder.layer.0.output.LayerNorm.adapter.layer_norm.bias True

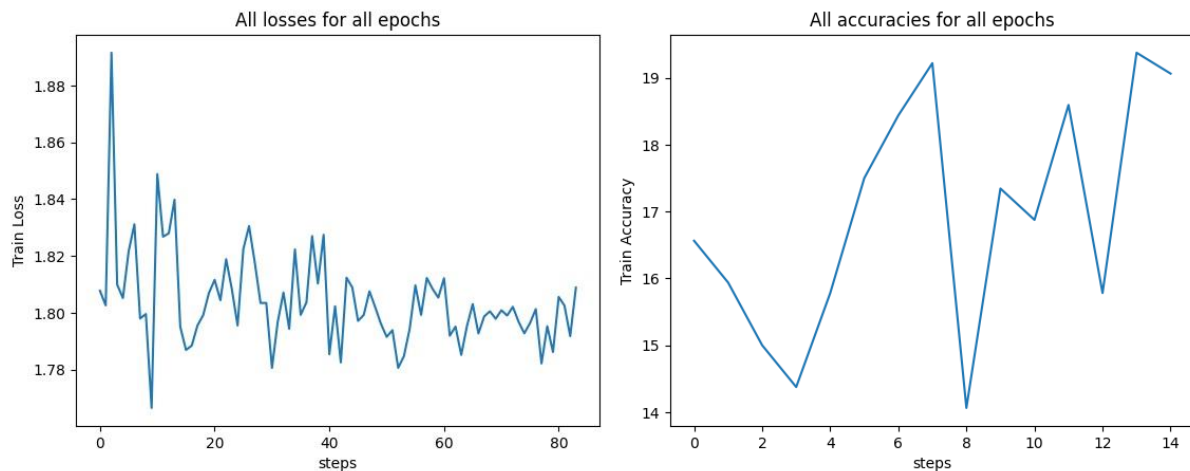
```

شکل ۴-۲- چگونگی گزاردیانه‌ها برای مدل Bert که همراه با Adapter طراحی شده است

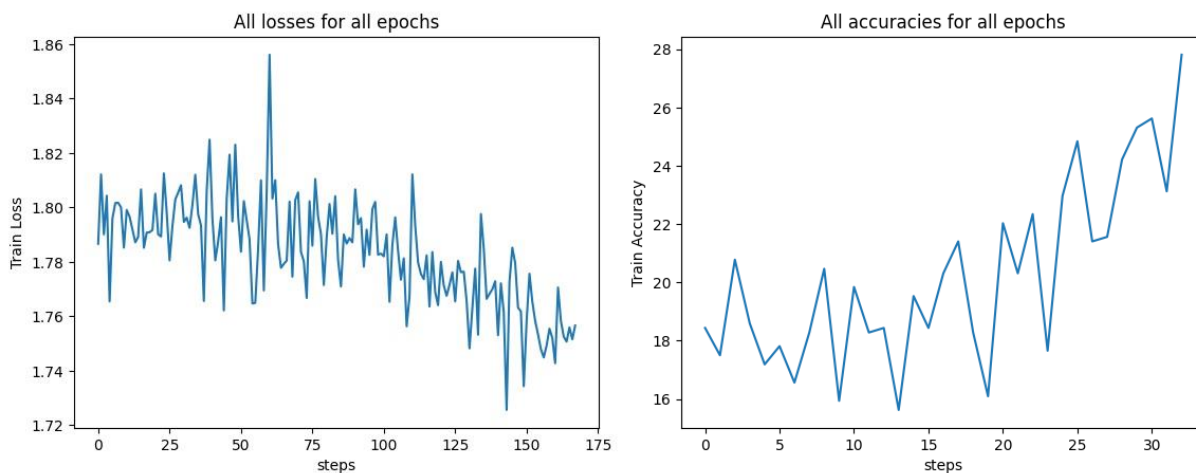
سپس، مدل کاملاً مشابه قسمت اول Train میشود و نتایج بر روی داده‌های Train به صورت زیر است:



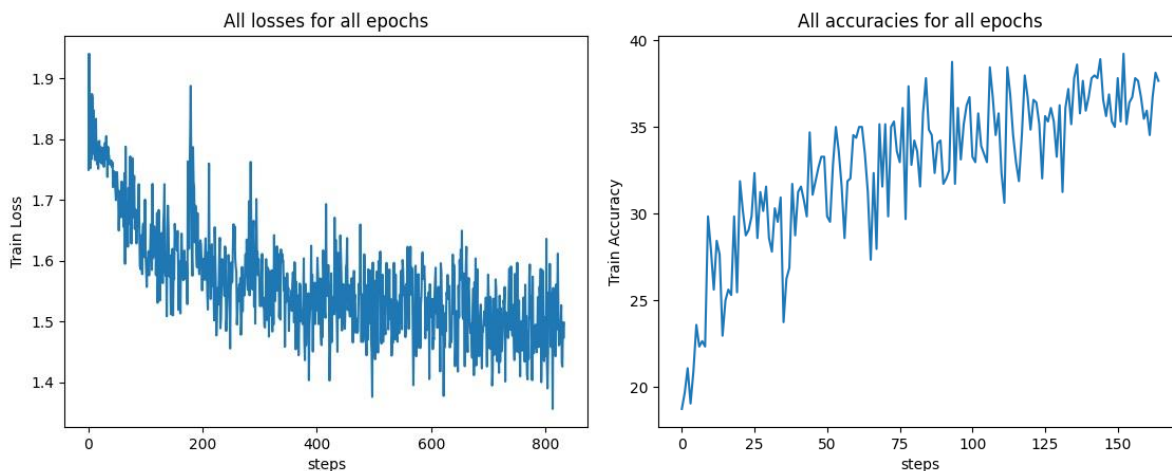
شکل ۵-۲- دقت و loss مربوط به مدل BERT با اضافه کردن Adapter توسط آموزش با ۱٪ داده‌های آموزشی



شکل ۲-۶- دقت و loss مربوط به مدل BERT با اضافه کردن Adapter توسط آموزش با ۵٪ داده‌های آموزشی

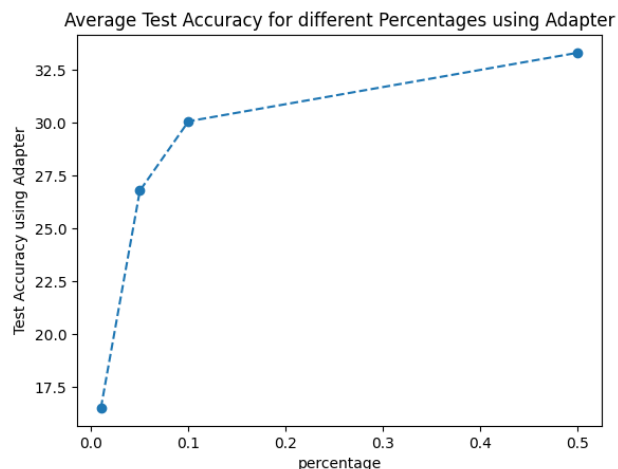


شکل ۲-۷- دقت و loss مربوط به مدل BERT با اضافه کردن Adapter توسط آموزش با ۱۰٪ داده‌های آموزشی



شکل ۲-۸- دقت و loss مربوط به مدل BERT با اضافه کردن Adapter توسط آموزش با ۵۰٪ داده‌های آموزشی

در نهایت میانگین دقت مدل BERT به همراه بلوک Adapter روی دیتاست تست به ازای درصد‌های مختلفی که شبکه از داده‌های لیبِل‌دار آموزش داده شده است، به صورت زیر حاصل می‌شود:



شکل ۹-۲- بررسی نتایج بر روی داده تست برای درصدهای مختلف

مانند سوال ۱ مشاهده می‌کنیم که با افزایش داده‌ها، دقت مدل برای داده‌های آزمون، افزایش می‌یابد.

در انتها، مقایسه‌ای برای زمان مورد نیاز برای شبیه سازی داده‌ها، برای سوال اول (که در آن Adapter حضور نداشت) و سوال دوم که در آن Adapter استفاده شده است، آورده‌ایم که اعداد سمت راست مربوط به حالت استفاده از Adapter و اعداد سمت چپ مربوط به سوال اول هستند که در فایل نوتبوک نیز، این اعداد قابل مشاهده هستند:

Not using Adapters	Using Adapters
Percentage: 1%	Percentage: 1%
Elapsed time is 61.07273 seconds	Elapsed time is 40.743 seconds
Percentage: 5%	Percentage: 5%
Elapsed time is 192.21087 seconds	Elapsed time is 200.72753 seconds
Percentage: 10%	Percentage: 10%
Elapsed time is 399.57117 seconds	Elapsed time is 399.87433 seconds
Percentage: 50%	Percentage: 50%
Elapsed time is 2026.0967 seconds	Elapsed time is 1878.85501 seconds

همانطور که مشخص است، زمان لازم برای شبیه سازی در حالت استفاده از Adapter، کاهش یافته است که علت آن، کمتر بودن تعداد وزنهای قابل یادگیری شبکه است.

روش دوم: پیاده سازی Adapter با استفاده از کتابخانه‌های آماده

لازم به ذکر است که متاسفانه این قسمت به نتیجه رسید!

در این قسمت از کتابخانه‌های آماده به صورت مقابل، پس از نصب Adapter_transformers استفاده شد [2]:

```
!pip -q install adapter-transformers
from transformers.adapters import AdapterType, AdapterConfig
print("Done!")
```

نکته‌ای که قابل توجه است، تطابق نسخه کتابخانه Transformer با Adapter می‌باشد. لذا باید نسخه دقیق Transformers نصب شود که این کار به صورت زیر انجام شد که ابتدا ماژول AutoModelWithHeads که مدلی جهت بیان یک Transformer همراه با Head های مختلف هست، و همچنین PfeifferConfig که مشخصات Adapter از جمله قسمت غیر خطی بودن و تعداد لایه‌هایی که می‌خواهیم قابل یادگیری باشند را در بر دارد، را وارد می‌کنیم و تنظیمات لازم را انجام می‌دهیم.

```
!pip install transformers==4.3.0
from transformers import AutoModelWithHeads, PfeifferConfig
model_name = "bert-base-uncased"
model = AutoModelWithHeads.from_pretrained(model_name)
# Create a new adapter configuration
adapter_config = PfeifferConfig(
    hidden_size=64,
    non_linearity="swish",
    reduction_factor=4,
    residual_before_ln=True,
)
# Add an adapter module to the model based on the configuration
adapter_name = "text_classification_adapter"
model.add_adapter(adapter_name, config=adapter_config)
```

چون مسئله ما، طبقه بندی است، نام adapter به صورت text_classification_adapter در نظر گرفته شد و پس از تنظیم Adapter، این ماژول به مدل اضافه می‌شود و مابقی مراحل کاملاً مشابه سوال اول پروژه است؛ این برنامه در برخی مواقع نمیتوانست اجرا شود!

رویکرد دیگر استفاده از کد زیر برای استفاده از Adapter است که در آن، لایه میانی Adapter که ۵۱۲ در نظر گرفته شده بود، قابل تنظیم است و مجدداً این قسمت با موفقیت رو به رو نشد!

```
# Install the required libraries and peft in Colab
!pip install --upgrade huggingface-hub==0.17.0
!pip install --upgrade tokenizers==0.14

# Install peft
!pip install peft

bert_model = BertModel.from_pretrained("bert-base-uncased")

m=512
adapter_name = "my_adapter"
adapter_config = {"adapter_dim": m}
```

```
bert_model.add_adapter(adapter_name, m)
```

۳. پیاده‌سازی شبکه GAN-BERT

استفاده از شبکه G_2 به عنوان Generator

در پیاده‌سازی این تمرین، به منظور استفاده از ظرفیت GPU بالاتر از Kaggle استفاده کردیم. مشکل Kaggle این است که پکیج gdown به صورت پیش فرض تعریف نشده است. لذا ابتدا پکیج آن را نصب می‌کنیم:

```
!pip install gdown
```

سپس کتابخانه‌های لازم را وارد می‌کنیم:

```
# Import libraries
import json
import tqdm
import torch
import random
import numpy as np
import pandas as pd
import torch.nn as nn
import torch.nn.functional as F
from transformers import AutoModel, AutoTokenizer, AutoConfig
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, SequentialSampler
```

در ادامه random_seed را طبق مطلوب مسئله برابر ۴۲ قرار می‌دهیم:

```
# Set seed
seed_val = 42
random.seed(seed_val)
np.random.seed(seed_val)
torch.manual_seed(seed_val)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed_val)
```

ادامه کد همانند سوال اول است که شامل تعریف تابع to_device و کلاس DeviceDataLoader، اضافه کردن device، دانلود و تشکیل دیتاست است. همچنین در این مسئله به منظور افزایش سرعت اجرای کد، ماکسیمم تعداد توکن ورودی به شبکه BERT را برابر ۶۴ قرار می‌دهیم:

```
# Dataset parameters
max_seq_length = 64
batch_size = 64

# Use Percentage% of the labeled data for training
Percentage = 0.5
```

در کد زیر، با استفاده از دستور sample، تعداد درصدی که نیاز داریم را به صورت تصادفی انتخاب می‌کنیم؛ که این درصد، ۵۰ است و این متغیرها را به عنوان داده‌های با لیبل در نظر می‌گیریم و آن‌ها را از تمام داده‌ها با استفاده از دستور drop، جدا می‌کنیم تا داده‌های بدون لیبل را نیز تشکیل دهیم.

```
df_train_for_ganbert = df_train.sample(frac = Percentage)
```

```
# Use the (1 - Percentage) remaining as unlabeled
```

```
df_unlabeled = df_train.drop(df_train_for_ganbert.index)
```

```
# Print labeled and unlabeled datasets shape
```

```
print('Labeled Data:',df_train_for_ganbert.shape)
```

```
print('Unlabeled Data:',df_unlabeled.shape)
```

```
# available labels in dataset
```

```
label_list = ['UNK', 'chatGPT', 'human', 'cohere', 'davinci', 'bloomz', 'dolly']
```

در این بخش برای داده‌های بدون لیبل، برچسب UNK را در نظر می‌گیریم که در ادامه، آن‌ها را برای محاسبه تابع هزینه D لحاظ خواهیم کرد. درواقع در ادامه به ازای درصد مشخصی از داده‌های لیبل دار که می‌خواهیم استفاده کنیم، یک لیبل جدید به نام UNK تعریف می‌کنیم و داده‌های بدون لیبل را به آن نسبت می‌دهیم. این کار موجب سهولت دسترسی به داده‌های بدون لیبل می‌شود.

```
# Set unknown label for unlabeled data
```

```
for i in df_unlabeled.index :
```

```
    df_unlabeled.at[i, "Label"] = "UNK"
```

در ادامه برای هریک از لیبل‌ها که به صورت string هستند، یک عدد اختصاص می‌دهیم و آن‌ها را در دیکشنری label_map ذخیره می‌کنیم؛ در واقع از این متغیر، در کلاس مجموعه داده، برای نگاشت لیبل‌ها از string به یک متغیر integer استفاده می‌کنیم.

```
# Define a label map dictionary for creating datasets
```

```
label_map = {}
```

```
for (i, label) in enumerate(label_list):
```

```
    label_map[label] = i
```

سپس از دیتاست اصلی تنها متن و لیبل آن‌ها را جدا کرده و کنار هم concatenate می‌کنیم و دیتاست جدید را تشکیل می‌دهیم. برای اینکار تابع get_datapoints را تعریف کرده و از روی آن، دیتاست داده‌های لیبل دار، بدون لیبل و تست را تشکیل می‌دهیم:

```
# A function for get datapoints from df
```

```
def get_datapoints(df):
```

```
    # A list to store the datapoints
```

```
    rows = []
```

```
    # Loop through rows
```

```
    for _, row in df.iterrows():
```

```
        rows.append((row['Text'], row['Label']))
```

```
return rows
```

```
# Apply the function and create examples from dfs
labeled_data = get_datapoints(df_train_for_ganbert)
unlabeled_data = get_datapoints(df_unlabeled)
test_data = get_datapoints(df_test)
```

حال tokenizer را از کتابخانه transformers روی مدل BERT تعریف می‌کنیم:

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

سپس در ادامه تابع create_dataset را تعریف می‌کنیم که با دریافت متن ورودی، label_map و label_mask دیتاست ایجاد می‌کند. متغیر label_mask لیبل دار بودن یا نبودن داده‌ها را مشخص می‌کند و label_map که در بخش قبل توضیح داده شد، به هر لیبل یک عدد بین ۱ تا ۷ نسبت می‌دهد که ۶ تای آن لیبل‌های اصلی و لیبل هفتم مربوط به داده‌های بدون لیبل یا UNK است. این تابع با دریافت آرگومان‌ها ابتدا متن‌ها را tokenize می‌کند و سپس بردار attention_mask را ایجاد می‌کند. این بردار درواقع برای توکن‌هایی غیر از توکن padding یا توکن 0 برابر 1 بوده و برای توکن‌های padding برابر 0 است. این بردار به مدل کمک می‌کند تا توکن‌های padding را نادیده بگیرد و در آموزش شبکه استفاده نکند. در انتهای این تابع تمامی آرایه‌ها به تانسور تبدیل شده و توسط دستور TensorDataset تبدیل به دیتاست می‌شوند:

```
# Main function for creating semi-supervised dataset
def create_dataset(input_examples, label_masks, label_map):

    # A list to save data and masks
    the_data = []

    # Loop through the datapoints and get data and the mask
    for idx, data in enumerate(input_examples):
        the_data.append((data, label_masks[idx]))

    input_ids = []
    attention_mask = []
    label_mask_array = []
    label_id_array = []

    # Tokenization
    for (text, label_mask) in tqdm.tqdm(the_data):
        encoded_sent = tokenizer.encode(
            text[0], add_special_tokens=True, max_length=max_seq_length,
            padding="max_length", truncation=True)

        input_ids.append(encoded_sent)
        label_id_array.append(label_map[text[1]])
        label_mask_array.append(label_mask)
```

```

# Attention to token
for sent in input_ids:
    att_mask = [int(token_id > 0) for token_id in sent]
    attention_mask.append(att_mask)

# Convert to Tensor
input_ids = torch.tensor(input_ids)
attention_mask = torch.tensor(attention_mask)
label_id_array = torch.tensor(label_id_array, dtype=torch.long)
label_mask_array = torch.tensor(label_mask_array)

# Make the TensorDataset
dataset = TensorDataset(input_ids, attention_mask, label_id_array, label_mask_array)

return dataset

```

حال کافیت دیتاست train را متشکل از دیتاهای لیبل دار و بدون لیبل ایجاد کنیم. همچنین دیتاست test را نیز ایجاد می کنیم:

```

# Create a semisupervised train datapoints
all_train_data = labeled_data + unlabeled_data

# The labeled dataset has mask = True
train_label_masks = np.ones(len(labeled_data), dtype=bool)

# The unlabeled dataset has mask = False
train_unlabel_masks = np.zeros(len(unlabeled_data), dtype=bool)

# Concat the masks
train_label_masks = np.concatenate([train_label_masks, train_unlabel_masks])

# Create trainset and
train_dataset = create_dataset(all_train_data, train_label_masks, label_map)

# Create train dataloader
train_dataloader = DataLoader(train_dataset, sampler = RandomSampler(train_dataset), batch_size
= batch_size)
train_dataloader=DeviceDataLoader(train_dataloader, device)

# The labeled dataset has mask = True
test_label_masks = np.ones(len(test_data), dtype=bool)

# Create testset
test_dataset = create_dataset(test_data, test_label_masks, label_map)

```

```
# Create test dataloader
test_dataloader = DataLoader(test_dataset, sampler =
SequentialSampler(test_dataset),batch_size = batch_size)
test_dataloader=DeviceDataLoader(test_dataloader,device)
```

با تعریف دیتاست به سراغ طراحی معماری شبکه می‌رویم. در این بخش در معماری شبکه generator از مدل G2 یا مدل BERT استفاده می‌کنیم. بنابراین کلاس‌های مدل generator و discriminator به صورت زیر تعریف می‌شوند.

معماری Generator، درواقع همان مدل Bert است که با توجه به Bag of Words ای که دریافت می‌کند، خروجی را تولید می‌کند. لازم به ذکر است که این خروجی شامل به علت تصادفی بودن کلمات ورودی Generator، باید توسط Discriminator، تقلبی اعلام شود.

```
# Create Generator class
class Generator(nn.Module):
    def __init__(self, bert_model, bag_of_words_dim):
        super(Generator, self).__init__()
        self.bert_model = bert_model

    def forward(self, bag_of_words):
        generated_output=self.bert_model(bag_of_words[0], attention_mask=bag_of_words[1])
        return generated_output
```

معماری Discriminator، طبق معماری مد نظر سوال پروژه طراحی شده است که در خروجی این کلاس، ۳ خروجی softmax، آخرین لایه خطی و همچنین خروجی لایه‌های feature ظاهر می‌شوند که در مورد این موارد در جلوتر توضیح داده می‌شود.

```
class Discriminator(nn.Module):
    def __init__(self,input_dim,hidden_layers, output_layer):
        super(Discriminator, self).__init__()
        self.Discriminator_Features=nn.Sequential( nn.Dropout(p=0.1),
                                                    nn.Linear(input_dim,hidden_layers[0]),
                                                    nn.LeakyReLU(0.2),
                                                    nn.Dropout(p=0.1))

        self.last_linear = nn.Linear(hidden_layers[0],output_layer)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        features = self.Discriminator_Features(x)
        last_linear_val = self.last_linear(features)
        output = self.softmax(last_linear_val)

        return features, last_linear_val, output
```


در شبکه discriminator به منظور انجام عمل feature matching با هدف جلوگیری از پدیده mode collapse خروجی آخرین لایه discriminator را با نام متغیر features در خروجی مدل قرار می‌دهیم.

حال تعداد نوروں‌های لایه مخفی شبکه discriminator، تعداد ابعاد خروجی مدل generator و ورودی discriminator را مشخص کرده و مدل BERT و شبکه‌های generator و discriminator را تعریف می‌کنیم.

```
# Set hidden size for G/D
hidden_size = 512
hidden_layers_generator = [hidden_size,hidden_size//2, hidden_size//2, hidden_size]
hidden_levels_discriminator = [hidden_size,hidden_size//2, hidden_size//2, hidden_size]

# Set noise and output dimansions
noise_dim = 100
output_layer = 768
bag_of_words_dim=output_layer

# Create G/D models
bert_model_for_generator = AutoModel.from_pretrained('bert-base-uncased')

generator_network = Generator(bert_model=bert_model_for_generator,
bag_of_words_dim=bag_of_words_dim)
# We have len(np.unique(all_labels_test)) classes for bert_model, and have 1 label for G and 1
label for unknown data.
```

لازم به ذکر است که خروجی Discriminator یک خروجی ۸ کلاسه است؛ زیرا داده‌ی ما دارای ۶ کلاس است و یک کلاس به صورت UNK برای داده‌های بدون برچسب در نظر گرفته شد و یک کلاس نیز برای تقلبی یا واقعی بودن داده ورودی Discriminator در نظر گرفته شد.

```
discriminator_network = Discriminator( input_dim = output_layer,hidden_layers =
hidden_levels_discriminator,output_layer = len(np.unique(all_labels_test)) + 2)
```

سپس تمام مدل‌های تعریف شده را توسط دستور to_device وارد GPU می‌کنیم:

```
# Load BERT
bert_model = AutoModel.from_pretrained("bert-base-uncased")

# Put everything in the GPU
generator_network=to_device(generator_network,device)
discriminator_network=to_device(discriminator_network,device)
bert_model=to_device(bert_model,device)
```

حال می‌خواهیم مدل را آموزش دهیم. مدل در ۳ اپیاک آموزش داده می‌شود. با توجه به اینکه بیشتر از دوبار نمی‌توان از backward مربوط به loss شبکه استفاده کرد، لذا پارامترهای شبکه BERT و discriminator را باهم ترکیب می‌کنیم تا یک بار برای Discriminator و Bert به طور همزمان، و یک بار نیز برای G، عمل Backward را انجام دهیم.

```
# Set number of epochs
num_train_epochs = 3

# Save training/val/test stats
training_stats = []

# List of models parameters for passing to optimizer
bert_params = [i for i in bert_model.parameters()]
discriminator_params = [v for v in discriminator_network.parameters()] + bert_params
generator_params = [v for v in generator_network.parameters()]

# Set optimization parameters
learning_rate_discriminator = 5e-5
learning_rate_generator = 5e-5
epsilon = 1e-8
```

حال با توجه به اینکه شبکه BERT که از آن فیچرهای 768 تایی استخراج می‌کنیم از ورودی تصادفی bag of words استفاده می‌کند، لذا لازم است ورودی‌های تصادفی از دیتاست آموزشی تولید شود. بنابراین تابع `bag_of_words_among_dataset` را تعریف می‌کنیم. این تابع با دریافت دیتاست ورودی، سائز بچ و ماکسیمم طول هر جمله توکن شده، به طور تصادفی از تمام بچ‌ها یک توکن را به صورت تصادفی انتخاب می‌کند و به عنوان bag of words به عنوان ورودی شبکه BERT generator استفاده خواهد شد:

```
def bag_of_words_among_dataset(train_dataset, batch_size, max_length):
    train_loader2 = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    train_loader2 = DeviceDataLoader(train_loader2, device)

    all_input_ids=torch.zeros((batch_size,max_length)).to(device).to(torch.long)
    all_attention_mask=torch.zeros((batch_size,max_length)).to(device).to(torch.long)

    counter=0
    for batch in train_loader2:

        counter=counter+1
        input_ids, attention_mask, _, _ = batch

        random_idx = torch.randint(0,input_ids.shape[1],(1,)).to(device).squeeze()
        all_input_ids[:,counter-1]=input_ids[:,random_idx]
        all_attention_mask[:,counter-1]=attention_mask[:,random_idx]
```

```

if(counter==batch_size):
    break

return all_input_ids, all_attention_mask

```

سپس نوع optimizer را مشخص می‌کنیم:

```

# Set optimizers
discriminator_optimizer = torch.optim.AdamW(discriminator_params,
lr=learning_rate_discriminator)
generator_optimizer = torch.optim.AdamW(generator_params, lr=learning_rate_generator)

```

لازم به ذکر است که پارامترهای شبکه discriminator در واقع ترکیب پارامترهای شبکه BERT و discriminator که کلاس آن را تعریف کرده بودیم می‌باشد. حال می‌خواهیم مدل را train کنیم:

```

# Loop through epochs
for epoch in range(0, num_train_epochs):
    train_loss_generator = 0

    train_loss_discriminator = 0
    # Set training mode
    bert_model.train()
    generator_network.train()
    discriminator_network.train()
    print("#####")
    print("This model is working for the train for the epoch",epoch+1)
    print("\n\n\n\n")

    # Loop through the batches
    for step, batch in tqdm.tqdm(enumerate(train_dataloader)):

        # Unpack the training batch
        input_ids, attention_mask, labels, masks = batch
        the_batch_size = input_ids.shape[0]

```

در آموزش شبکه ابتدا متن ورودی هر بچ وارد شبکه BERT می‌شود و آخرین خروجی آن که یک بردار ۷۶۸ تایی است استخراج می‌شود:

```

# Encode real data in the BERT
bert_out = bert_model(input_ids, attention_mask=attention_mask)
features_bert = bert_out[-1]

```

سپس توسط تابع `bag_of_words_among_dataset` یک مجموعه `bag_of_words` متشکل از توکن‌های رندوم تشکیل می‌شود که وارد شبکه `generator` می‌شود سپس آخرین خروجی آن به عنوان فیچرهای خروجی شبکه `generator` در نظر گرفته می‌شود:

```
# Create noise to feed to the generator
bag_of_words = bag_of_words_among_dataset(train_dataset, batch_size, input_ids.shape[1])

# Pass bag of words through the BagOfWordsGenerator
output_generator = generator_network(bag_of_words)
features_generator = torch.squeeze(output_generator[0][:, -1, :])
```

در ادامه خروجی شبکه `generator` با شبکه `BERT` الحاق می‌شود تا وارد شبکه `discriminator` شود:

```
# Feed the output of the bert and the generator to discriminator
discriminator_input = torch.cat([features_bert, features_generator], dim=0)

# Get output of the discriminator
features, logits, probs = discriminator_network(discriminator_input)
```

شبکه `discriminator` خروجی‌های `features`، `logits` و `probs` را به ما می‌دهد. خروجی `features` از خروجی آخرین لایه، `FC` گرفته شده است. خروجی `logits` و `probs` نیز شامل خروجی مدل قبل و بعد از اعمال `softmax` هستند. سپس خروجی شبکه `discriminator` به ازای هر ورودی که از شبکه `BERT` و `generator` وارد می‌شود خروجی‌های آن جدا شده و در متغیرهای جداگانه ذخیره می‌شوند:

```
# Separate the output of discriminator for the real and fake
features_list = torch.split(features, the_batch_size)
D_real_features = features_list[0]
D_fake_features = features_list[1]

logits_list = torch.split(logits, the_batch_size)
D_real_logits = logits_list[0]
D_fake_logits = logits_list[1]

probs_list = torch.split(probs, the_batch_size)
D_real_probs = probs_list[0]
D_fake_probs = probs_list[1]
```

حال براساس این اطلاعات، `loss` شبکه را محاسبه می‌کنیم. ابتدا برای بدست آوردن `loss` مربوط به شبکه `generator` که برابر با $L_g = L_{g_{feature\ matching}} + L_{g_{unsup}}$ است را محاسبه می‌کنیم. ترم اول مربوط به عمل `feature matching` به منظور جلوگیری از پدیده `mode collapse` محاسبه می‌شود تا مطمئن باشیم شبکه `generator` ورودی‌های متنوعی مطابق با داده‌های واقعی تولید می‌کند. ترم دوم آن نیز براساس عملکرد شبکه `discriminator` از روی تشخیص واقعی بودن یا فیک بودن داده تولیدی شبکه `generator` محاسبه می‌شود:

```
# Generator LOSS
g_loss_d = -1 * torch.mean(torch.log(1 - D_fake_probs[:, -1] + epsilon))

g_feat_reg = torch.mean(torch.pow(
    torch.mean(D_real_features, dim=0)
    - torch.mean(D_fake_features, dim=0), 2))

all_losses_for_G = g_loss_d + g_feat_reg
```

در ادامه شبکه discriminator را نیز محاسبه می‌کنیم که شامل جمع دو ترم مختلف است: $L_D = L_{D_{sup}} + L_{D_{unsup}}$

با توجه به اینکه از label_mask برای مشخص کردن وجود یا عدم وجود لیبل در داده‌های ورودی استفاده می‌کنیم، لذا ترم $L_{D_{sup}}$ برای داده‌های بدون لیبل mask شده و برابر صفر می‌شود. ترم دوم $L_{D_{unsup}}$ اما فقط هزینه فیک یا واقعی بودن داده ورودی را محاسبه می‌کند و برای همه داده‌ها یکسان است و درواقع یک binary cross entropy ساده می‌باشد. در این کد $L_{D_{unsup}}$ شامل دو ترم $D_L_unsupervised1U$ و $D_L_unsupervised2U$ می‌باشد که $D_L_unsupervised1U$ هزینه ناشی از تشخیص فیک بودن داده‌های واقعی و $D_L_unsupervised2U$ هزینه ناشی از تشخیص واقعی بودن داده‌های فیک می‌باشد.

در واقع، ارقام پیشبینی توسط Discriminator را از Softmax عبور می‌دهیم و یک بار نیز آن‌ها را One-Hot می‌کنیم تا بتوانیم از ضرب آنها و drop کردن لیبلهایی که دارای ماسک False هستند، هزینه Discriminator برای تقلبی یا واقعی بودن داده استفاده کنیم و پس از آن، تعداد لیبلها را با استفاده از دستور numel بدست می‌آوریم تا با تقسیم هزینه بدست آمده بر تعداد آن‌ها، loss برای تشخیص صحیح لیبلها را محاسبه کنیم.

```
# Discriminator LOSS
logits = D_real_logits[:, 0:-1]
log_probs = F.log_softmax(logits, dim=-1)

# The Loss for unlabeled data is masked out
label2one_hot = torch.nn.functional.one_hot(labels, len(label_list))

per_example_loss = -torch.sum(label2one_hot * log_probs, dim=-1)

per_example_loss = torch.masked_select(per_example_loss, masks.to(device))

labeled_example_count = per_example_loss.type(torch.float32).numel()

if labeled_example_count == 0:
    D_L_Supervised = 0
else:
    D_L_Supervised = torch.div(
        torch.sum(per_example_loss.to(device)), labeled_example_count)
```

```

D_L_unsupervised1U = -1 * torch.mean(
    torch.log(1 - D_real_probs[:, -1] + epsilon))

D_L_unsupervised2U = -1 * torch.mean(
    torch.log(D_fake_probs[:, -1] + epsilon))

all_losses_for_D= D_L_Supervised + D_L_unsupervised1U + D_L_unsupervised2U

```

با محاسبه loss مربوط به هردو شبکه generator و discriminator، backward شبکه را انجام داده و پارامترهای دو شبکه را آپدیت می‌کنیم. سپس میانگین دقت شبکه را نیز در یک batch بدست می‌آوریم. لازم به ذکر است که چون ۲ عمل Backward داریم، یکی از آنها باید عبارت retain_graph=True را دربر داشته باشد.

```

# Reset gradients
generator_optimizer.zero_grad()
discriminator_optimizer.zero_grad()

# Backward pass
all_losses_for_G.backward(retain_graph=True)
all_losses_for_D.backward()

# Update weights
generator_optimizer.step()
discriminator_optimizer.step()

# Save the losses to report
train_loss_generator += all_losses_for_G.item()
train_loss_discriminator += all_losses_for_D.item()

if (step%20==0):
    print("step", step, "from", 1+len(train_dataset)//batch_size)

# Calculate the average loss over the batches.
avg_train_loss_g = train_loss_generator / len(train_dataloader)
avg_train_loss_d = train_loss_discriminator / len(train_dataloader)

print("Average training generator loss:", avg_train_loss_g)
print("Average training discriminator loss:", avg_train_loss_d)

print("#####")
print("This model is working for the test for the epoch", epoch+1)
print("\n\n\n\n")

```

در ادامه برای داده تست نیز این مراحل را بدون آپدیت وزن‌های شبکه و عمل backward انجام داده و دقت مدل را گزارش می‌دهیم:

```
# Set validation mode
bert_model.eval()
discriminator_network.eval()
generator_network.eval()

# Tracking variables
total_test_accuracy = 0

total_test_loss = 0
nb_test_steps = 0

all_preds = []
all_labels_ids = []

# Define Loss function
nll_loss = torch.nn.CrossEntropyLoss(ignore_index=-1)

# Loop through test patches
for batch in test_dataloader:

    input_ids, attention_mask, labels, _ = batch
    # No grad block
    with torch.no_grad():
        bert_out = bert_model(input_ids, attention_mask=attention_mask)

        features_bert = bert_out[-1]

        _, logits, probs = discriminator_network(features_bert)

        filtered_logits = logits[:,0:-1]

        # Accumulate the test loss.
        total_test_loss += nll_loss(filtered_logits, labels)

    # Accumulate the predictions and the input labels
    _, preds = torch.max(filtered_logits, 1)
    all_preds += preds.detach().cpu()
    all_labels_ids += labels.detach().cpu()

# Report the final accuracy for this validation run.
all_preds = torch.stack(all_preds).numpy()
all_labels_ids = torch.stack(all_labels_ids).numpy()
```

```

test_accuracy = np.sum(all_preds == all_labels_ids) / len(all_preds)
print("Test Accuracy for epoch",epoch+1," is: ",test_accuracy)

# Calculate the average loss over all of the batches.
avg_test_loss = total_test_loss / len(test_dataloader)
avg_test_loss = avg_test_loss.item()

# Print validation stats
print("Average training generator loss:",avg_train_loss_g)

# Store stats from this epoch.
training_stats.append({'epoch': epoch + 1,'Test Accuracy': test_accuracy})

```

در آخر اطلاعات مربوط به loss و دقت مدل در دیتاست تست را در یک فایل با فرمت json ذخیره می‌کنیم. دلیل استفاده از این روش بجای استفاده از یک حلقه for برای آموزش شبکه روی درصد های مختلف داده‌های لیبل دار این است که در جریان آموزش شبکه و دریافت نتایج در پلتفرم Kaggle با قطع شدن احتمالی اینترنت، اطلاعات از بین نروند. لذا برای هر درصدی به طور جداگانه شبکه را آموزش دادیم و فایل‌های json توسط نوت بوک Final_Results_for_Part_3_and_Part_4_Network.ipynb که در فایل‌های پروژه ضمیمه شده است، نمودار نهایی را رسم می‌کنیم.

استفاده از شبکه G_1 به عنوان Generator

با توجه به توضیحاتی که در بخش قبل داده شد، در این بخش تنها به تفاوت‌های پیاده سازی نسبت به بخش قبل اشاره می‌شود:

۱- معماری شبکه generator از معماری BERT پیروی نکرده و کلاس آن طبق صورت سوال پروژه به صورت زیر تعریف می‌شود:

```

# Create Generator class
class Generator(nn.Module):
    def __init__(self, noise_dim,hidden_layers, output_layer):
        super(Generator, self).__init__()
        self.Generator_Network=nn.Sequential(
            nn.Linear(noise_dim, hidden_layers[0]), nn.LeakyReLU(0.2),
            nn.Dropout(p=0.1), nn.Linear(hidden_layers[0], output_layer))

    def forward(self,x):
        output= self.Generator_Network(x)
        return output

```


۲- تابع `bag_of_words_among_dataset` حذف شده و بجای `bag of word`، نویز با تعداد ابعاد $batch\ size \times 100$ وارد شبکه `generator` می‌شود:

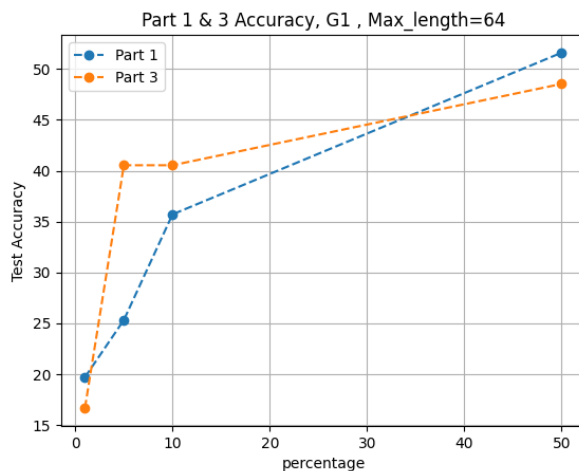
```
noise_dim = 100
...
# Create noise to feed to the generator
noise = torch.randn(real_batch_size, noise_dim, device=device)

# Gnerate Fake data
gen_rep = generator(noise)
```

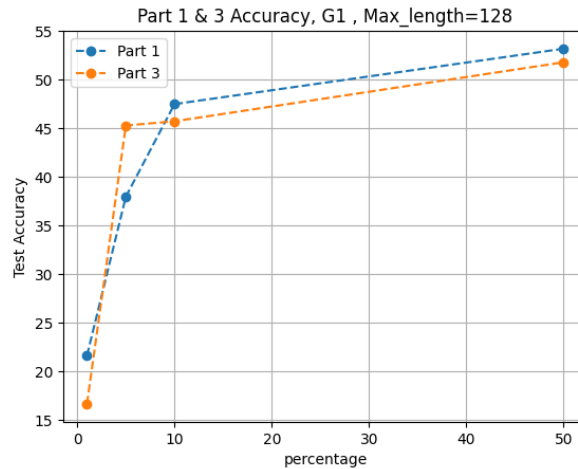
باقی موارد همانند پیاده سازی این مدل توسط شبکه `G2` است.

نمودارهای مربوط به مقایسه قسمت اول پروژه و قسمت سوم آن

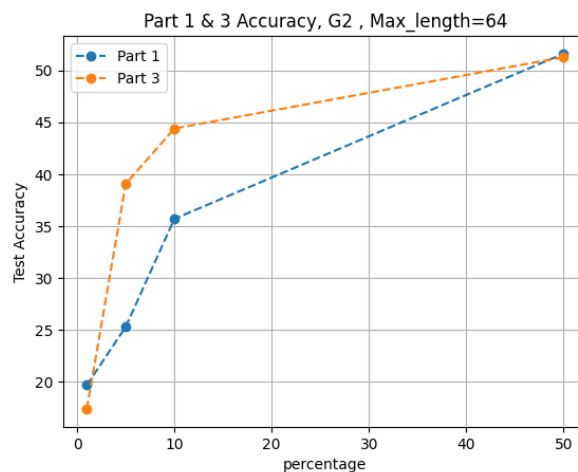
شکل‌های زیر، نتایج نهایی بخش سوم پروژه هستند:



شکل ۳-۱- مقایسه قسمت اول و سوم پروژه برای استفاده از شبکه `G1` به عنوان `Generator` با در نظر گرفتن `Max_length=64`



شکل ۳-۲- مقایسه قسمت اول و سوم پروژه برای استفاده از شبکه G1 به عنوان Generator با در نظر گرفتن Max_length=128

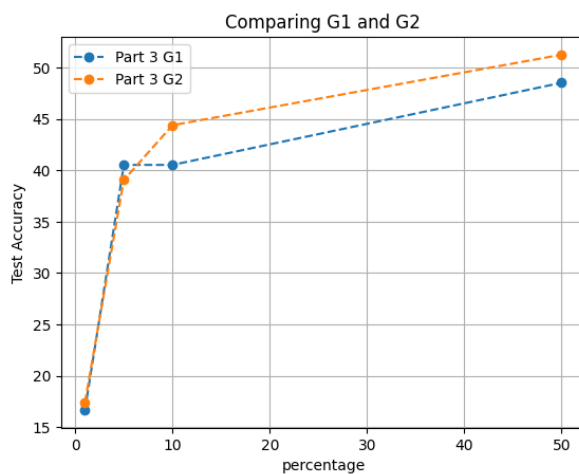


شکل ۳-۳- مقایسه قسمت اول و سوم پروژه برای استفاده از شبکه G2 به عنوان Generator با در نظر گرفتن Max_length=64

همانطور که از شکل‌های ۱-۳ الی ۳-۳ مشخص است، به طور کلی، شبکه GAN_BERT توانایی بیشتری برای تشخیص کلاس متن‌ها دارد؛ زیرا شبکه G حضور دارد و می‌تواند به شبکه BERT کمک کند و به نوعی با شبکه Discriminator در رقابت است و باعث تقویت شبکه Discriminator می‌شود! ضمناً طبق شکل‌های ۱-۳ و ۲-۳، با افزایش Max_length، نتیجه هر دو قسمت اول و سوم پروژه بهبود می‌یابد؛ زیرا به داده‌های برجسته‌تر دسترسی داریم که از داده‌های بدون برجسته‌کاری بیشتری دارند.

همچنین، طبق شکل ۴-۳، شبکه G2 عملکرد بهتری از شبکه G1 داشته است؛ زیرا می‌دانیم که در یک شبکه GAN، بلوک G وظیفه‌ی تخمین زدن توزیع داده‌ها را دارد و چون داده‌ها توسط مدل Bert تولید می‌شوند، هنگامی که معماری شبکه Generator

یک Bert است، بهتر می‌تواند توزیع داده‌ها را تخمین بزند و Discriminator از آن، فیدبک قوی‌تری می‌گیرد و سبب تقویت بیشتر Discriminator می‌شود.



شکل ۳-۴- مقایسه شبکه‌های G_1 و G_2 برای قسمت سوم پروژه

۴. بهبود شبکه GAN-BERT

در این قسمت، ابتدا به بهبود شبکه GAN_BERT برای حالت استفاده از G_1 به عنوان Generator می‌پردازیم؛ زیرا شبیه سازی بر روی این شبکه ساده تر بوده است و زمان کمتری برای شبیه سازی نیاز داشته است و در انتها، موارد انجام شده برای کسب نتیجه بهتر برای حالت استفاده از G_2 را شرح خواهیم داد.

آزمایشات انجام شده جهت بهبود عملکرد شبکه GAN-BERT با معماری G_1

در این قسمت ابتدا با توجه به مشاهدات مدل های مختلف را آزمایش کرده ایم و با تعداد ایپاک محدود نتایج را مقایسه کرده ایم، در نهایت از میان آنها بهترین مدل را انتخاب و با ماکسیمم طول جمله ۲۵۶ و تعداد ایپاک ۱۰ آموزش داده ایم که نتایج بهترین مدل در انتهای این بخش گزارش شده است.

نکته: طبق آزمایش های انجام شده، بهترین نتیجه مربوط به آزمایش دهم است.

آزمایش اول

- ساختار G_1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل uncased برای BERT
- ماکسیمم طول رشته ها: ۶۴
- سایز بسته (Batch size): ۱۲۸

- بهترین دقت تست به دست آمده در ۳ ایپاک : ۴۸,۴٪

نتیجه بالا نشان میدهد حدود ۳ درصد دقت نسبت به حالت مشابه در سوال سوم پروژه بهبود عملکرد داشته ایم.

```
# Print G architectures
print(generator.parameters())

<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.1, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
)>
```

شکل ۴-۱- معماری آزمایش اول

==== Epoch 1 / 3 ====	==== Epoch 2 / 3 ====	==== Epoch 3 / 3 ====
Training...	Training...	Training...
200it [03:46, 1.13s/it]	200it [03:46, 1.13s/it]	200it [03:46, 1.13s/it]
Batch 200 of 555. Elapsed: 0:03:46.	Batch 200 of 555. Elapsed: 0:03:46.	Batch 200 of 555. Elapsed: 0:03:47.
400it [07:32, 1.13s/it]	400it [07:32, 1.14s/it]	400it [07:33, 1.14s/it]
Batch 400 of 555. Elapsed: 0:07:32.	Batch 400 of 555. Elapsed: 0:07:33.	Batch 400 of 555. Elapsed: 0:07:33.
555it [10:27, 1.13s/it]	555it [10:28, 1.13s/it]	555it [10:28, 1.13s/it]
Average training loss generator: 0.626 Average training loss discriminator: 2.264 Training epoch took: 0:10:28	Average training loss generator: 0.711 Average training loss discriminator: 1.546 Training epoch took: 0:10:28	Average training loss generator: 0.707 Average training loss discriminator: 1.256 Training epoch took: 0:10:29
Running Test...	Running Test...	Running Test...
Test Accuracy: 0.449 Test Loss: 1.452 Test took: 0:00:05	Test Accuracy: 0.445 Test Loss: 1.568 Test took: 0:00:05	Test Accuracy: 0.484 Test Loss: 1.785 Test took: 0:00:05

شکل ۴-۲- نتایج آزمایش اول

آزمایش دوم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالساز LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer)

در ورودی هر لایه

- ۵۰ درصد داده ها برچسب دار هستند

- استفاده از مدل cased برای BERT

- ماکسیمم طول رشته ها : ۲۵۶

- سایز بسته (Batch size) : ۳۲

- دقت تست به دست آمده در ۳ ایپاک اول به شرح زیر است :

- ایپاک اول : ۵۴٪ ایپاک دوم : ۴۹,۲٪ ایپاک سوم : ۶۱,۱٪ ایپاک چهارم : ۵۳,۲٪ ایپاک پنجم : ۶۱,۸٪

- ایپاک ششم : ۵۶,۶٪ ایپاک هفتم : ۵۳,۴٪ ایپاک هشتم : ۵۳,۷٪ ایپاک نهم : ۵۷,۱٪ ایپاک دهم : ۵۳,۴٪

همانطور که انتظار داشتیم با افزایش طول جملات عملکرد مدل بهتر شد. این آزمایش مشابه آزمایش اول است با این تفاوت که از مدل cased برای BERT استفاده شده است و طول جملات از ۶۴ به ۲۵۶ رسیده است. بهترین نتیجه در ایپاک پنجم به دست آمده است که برابر با ۶۱٫۸٪ می باشد. بعد از ایپاک پنجم عملکرد مدل کاهش یافته است که بنظر میرسد مدل دچار اورفیتینگ شده است. انجام این آزمایش در مجموع هفت ساعت و نیم به طول انجامید.

Average training loss generator: 0.682 Average training loss discriminator: 1.481 Training epoch took: 0:44:57	Average training loss generator: 0.706 Average training loss discriminator: 1.031 Training epoch took: 0:44:58	Average training loss generator: 0.705 Average training loss discriminator: 0.917 Training epoch took: 0:45:04
Running Test... Test Accuracy: 0.540 Test Loss: 1.552 Test took: 0:00:23	Running Test... Test Accuracy: 0.492 Test Loss: 2.085 Test took: 0:00:23	Running Test... Test Accuracy: 0.611 Test Loss: 1.642 Test took: 0:00:23
Average training loss generator: 0.703 Average training loss discriminator: 0.864 Training epoch took: 0:45:05	Average training loss generator: 0.702 Average training loss discriminator: 0.828 Training epoch took: 0:44:58	Average training loss generator: 0.702 Average training loss discriminator: 0.805 Training epoch took: 0:45:00
Running Test... Test Accuracy: 0.532 Test Loss: 2.157 Test took: 0:00:23	Running Test... Test Accuracy: 0.618 Test Loss: 2.124 Test took: 0:00:23	Running Test... Test Accuracy: 0.566 Test Loss: 2.508 Test took: 0:00:23
Average training loss generator: 0.701 Average training loss discriminator: 0.793 Training epoch took: 0:45:02	Average training loss generator: 0.701 Average training loss discriminator: 0.794 Training epoch took: 0:45:04	
Running Test... Test Accuracy: 0.537 Test Loss: 2.745 Test took: 0:00:23	Running Test... Test Accuracy: 0.571 Test Loss: 2.275 Test took: 0:00:23	
Average training loss generator: 0.700 Average training loss discriminator: 0.768 Training epoch took: 0:45:05	Average training loss generator: 0.701 Average training loss discriminator: 0.798 Training epoch took: 0:45:02	
Running Test... Test Accuracy: 0.534 Test Loss: 2.731 Test took: 0:00:23	Running Test... Test Accuracy: 0.534 Test Loss: 2.681 Test took: 0:00:23	

شکل ۳-۴- نتایج آزمایش دوم

```
# Print G architectures
print(generator.parameters())

<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.1, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
)>
```

شکل ۴-۴- معماری آزمایش دوم

آزمایش سوم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالساز LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه

- ساختار D : اضافه کردن سه لایه خطی (FC) با تابع فعالساز LeakyRelu و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل uncased برای BERT
- ماکسیمم طول رشته ها : ۶۴
- سایز بسته (Batch size) : 128
- بهترین دقت تست به دست آمده در سه ایپاک : ۴۷,۶٪

همانطور که مشاهده می شود نسبت به آزمایش اول صرفا مدل تمیزدهنده D را با اضافه کردن لایه های خطی و نرمالیزشن عمیق تر کرده ایم که موجب کاهش دقت شد.

```
<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.1, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
)
.....
<bound method Module.parameters of Discriminator(
  (Discriminator_Features): Sequential(
    (0): Dropout(p=0.1, inplace=False)
    (1): Linear(in_features=768, out_features=512, bias=True)
    (2): LeakyReLU(negative_slope=0.2)
    (3): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): Linear(in_features=512, out_features=256, bias=True)
    (5): LeakyReLU(negative_slope=0.2)
    (6): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): Linear(in_features=256, out_features=128, bias=True)
    (8): LeakyReLU(negative_slope=0.2)
    (9): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): Linear(in_features=128, out_features=32, bias=True)
    (11): LeakyReLU(negative_slope=0.2)
    (12): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): Dropout(p=0.1, inplace=False)
  )
)
```

شکل ۴-۵- معماری های آزمایش سوم

==== Epoch 3 / 3 ==== Training... 200it [03:47, 1.14s/it] Batch 200 of 555. Elapsed: 0:03:47. 400it [07:34, 1.14s/it] Batch 400 of 555. Elapsed: 0:07:34. 555it [10:30, 1.14s/it] Average training loss generetor: 0.616 Average training loss discriminator: 1.188 Training epoch took: 0:10:30 Running Test... Test Accuracy: 0.451 Test Loss: 2.299 Test took: 0:00:06	==== Epoch 1 / 3 ==== Training... 200it [03:46, 1.13s/it] Batch 200 of 555. Elapsed: 0:03:47. 400it [07:33, 1.13s/it] Batch 400 of 555. Elapsed: 0:07:34. 555it [10:29, 1.13s/it] Average training loss generetor: 0.535 Average training loss discriminator: 1.889 Training epoch took: 0:10:30 Running Test... Test Accuracy: 0.474 Test Loss: 1.852 Test took: 0:00:05	==== Epoch 2 / 3 ==== Training... 200it [03:47, 1.14s/it] Batch 200 of 555. Elapsed: 0:03:47. 400it [07:34, 1.14s/it] Batch 400 of 555. Elapsed: 0:07:34. 555it [10:30, 1.14s/it] Average training loss generetor: 0.595 Average training loss discriminator: 1.408 Training epoch took: 0:10:30 Running Test... Test Accuracy: 0.476 Test Loss: 2.058 Test took: 0:00:05
---	---	---

شکل ۴-۶- نتایج آزمایش سوم

آزمایش چهارم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- افزایش مقدار احتمال تمام Dropout های استفاده شده در ساختار G1 و D به $p=0.5$ برای کاهش اورفیتینگ مشاهده شده در آزمایشات قبلی
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیمم طول رشته ها : ۱۲۸
- سایز بسته (Batch size) : 128
- دقت تست به دست آمده به شرح زیر است :
- ایپاک اول : ۵۰٫۴٪ ایپاک دوم : ۵۵٪ ایپاک سوم : ۵۶٫۶٪ ایپاک چهارم : ۵۴٫۳٪ ایپاک پنجم : ۵۵٫۷٪

همانطور که مشاهده میشود با افزایش طول جملات، افزایش اثر **Dropout**، و استفاده از مدل **cased** برای BERT عملکرد مدل روی داده های تست بهتر شده است و نسبت به آزمایش اول ۵٫۳٪ درصد بهبود عملکرد داشته ایم.

```
*****
<bound method Module.parameters of Discriminator(
  (Discriminator_Features): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=768, out_features=512, bias=True)
    (2): LeakyReLU(negative_slope=0.2)
    (3): Dropout(p=0.5, inplace=False)
  )
)

<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.5, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
)
```

شکل ۷-۴- معماری های آزمایش چهارم

Epoch [1 / 10]	Epoch [2 / 10]	Epoch [3 / 10]
200it [07:27, 2.24s/it]	200it [07:26, 2.23s/it]	200it [07:26, 2.24s/it]
Batch 200 of 555. Elapsed: 0:07:28.	Batch 200 of 555. Elapsed: 0:07:27.	Batch 200 of 555. Elapsed: 0:07:27.
400it [14:56, 2.24s/it]	400it [14:53, 2.23s/it]	400it [14:53, 2.23s/it]
Batch 400 of 555. Elapsed: 0:14:56.	Batch 400 of 555. Elapsed: 0:14:54.	Batch 400 of 555. Elapsed: 0:14:53.
555it [20:43, 2.24s/it]	555it [20:39, 2.23s/it]	555it [20:39, 2.23s/it]
Average training loss generator: 0.586 Average training loss discriminator: 2.074 Training epoch took: 0:20:43	Average training loss generator: 0.705 Average training loss discriminator: 1.243 Training epoch took: 0:20:39	Average training loss generator: 0.701 Average training loss discriminator: 1.032 Training epoch took: 0:20:39
Test: Test Accuracy: 0.504 Test Loss: 1.483 Test took: 0:00:11	Test: Test Accuracy: 0.550 Test Loss: 1.395 Test took: 0:00:11	Test: Test Accuracy: 0.566 Test Loss: 1.622 Test took: 0:00:11
Epoch [4 / 10]	Epoch [5 / 10]	
200it [07:26, 2.23s/it]	200it [07:26, 2.23s/it]	
Batch 200 of 555. Elapsed: 0:07:27.	Batch 200 of 555. Elapsed: 0:07:27.	
400it [14:53, 2.23s/it]	400it [14:53, 2.23s/it]	
Batch 400 of 555. Elapsed: 0:14:53.	Batch 400 of 555. Elapsed: 0:14:53.	
555it [20:39, 2.23s/it]	555it [20:39, 2.23s/it]	
Average training loss generator: 0.698 Average training loss discriminator: 0.918 Training epoch took: 0:20:39	Average training loss generator: 0.696 Average training loss discriminator: 0.841 Training epoch took: 0:20:39	
Test: Test Accuracy: 0.543 Test Loss: 1.968 Test took: 0:00:11	Test: Test Accuracy: 0.557 Test Loss: 2.088 Test took: 0:00:11	

شکل ۴-۸- نتایج آزمایش چهارم

آزمایش پنجم

- ساختار G1: اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- اضافه کردن تابع فعالساز Tanh به خروجی مدل G1
- افزایش مقدار احتمال تمام Dropout های استفاده شده در ساختار G1 و D به $p=0.5$ برای کاهش اورفیتینگ مشاهده شده در آزمایشات قبلی
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیسم طول رشته ها: ۱۲۸
- سایز بسته (Batch size): 128
- دقت تست به دست آمده ایپاک اول: ۵۴٫۶٪

همانطور که مشاهده میشود نسبت به آزمایش شماره چهارم صرفاً یک تابع فعالسازی Tanh به ساختار G1 اضافه کرده ایم و تمامی موارد دیگر یکسان میباشد. که این تغییر کوچک موجب افزایش ۴٫۶٪ بر روی نمونه های تست شد.


```

<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.5, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
  (tanh): Tanh()
)

```

شکل ۹-۴- معماری آزمایش پنجم

```

===== Epoch 1 / 10 =====
Training...

200it [14:27, 4.36s/it]
    Batch 200 of 555. Elapsed: 0:14:28.

400it [28:59, 4.36s/it]
    Batch 400 of 555. Elapsed: 0:28:59.

555it [40:15, 4.35s/it]

Average training loss generetor: 0.523
Average training loss discriminator: 2.047
Training epoch took: 0:40:16

Running Test...
Test Accuracy: 0.546
Test Loss: 1.340
Test took: 0:00:23

```

شکل ۱۰-۴- نتایج آزمایش پنجم

آزمایش ششم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- اضافه کردن تابع فعالساز Tanh به خروجی مدل G1
- افزایش ابعاد نويز ورودی از ۱۰۰ به ۲۰۰
- افزایش مقدار احتمال تمام Dropout های استفاده شده در ساختار G1 و D به $p=0.5$ برای کاهش اورفیتینگ مشاهده شده در آزمایشات قبلی
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیمم طول رشته ها : ۱۲۸
- سایز بسته (Batch size) : 128
- دقت تست به دست آمده ایپاک اول : ۵۲,۷٪

همانطور که مشاهده میشود نسبت به آزمایش شماره پنجم صرفا ابعاد نويز گوسی ورودی دوبرابر شده است و دیگر موارد بدون تغییر هستند. در این آزمایش مشاهده کردیم که این مورد باعث کاهش عملکرد مدل از ۵۴,۶٪ (آزمایش پنجم) به ۵۲,۷٪ شده است.

```

<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=200, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.5, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
  (tanh): Tanh()
)

```

شکل ۱۱-۴- معماری آزمایش ششم

```

==== Epoch 1 / 10 ====
Training...
200it [14:17,  4.31s/it]
  Batch    200 of    555.   Elapsed: 0:14:18.
400it [28:40,  4.30s/it]
  Batch    400 of    555.   Elapsed: 0:28:40.
555it [39:47,  4.30s/it]

Average training loss generator: 0.517
Average training loss discriminator: 2.062
Training epoch took: 0:39:47

Running Test...
Test Accuracy: 0.527
Test Loss: 1.348
Test took: 0:00:23

```

کل ۱۲-۴- نتایج آزمایش ششم

آزمایش هفتم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- اضافه کردن تابع فعالساز Tanh به خروجی مدل G1
- کاهش ابعاد نویز ورودی از ۱۰۰ به ۵۰
- افزایش مقدار احتمال تمام Dropout های استفاده شده در ساختار G1 و D به $p=0.5$ برای کاهش اورفیتینگ مشاهده شده در آزمایشات قبلی
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیمم طول رشته ها : ۱۲۸
- سایز بسته (Batch size) : 128
- دقت تست به دست آمده ایپاک اول : ۴۷,۷٪

همانطور که مشاهده میشود نسبت به آزمایش شماره ششم بجای افزایش ابعاد نويز گوسی ورودی انرا کاهش داده ایم و از مقدار پیشفرض ۱۰۰ به ۵۰ تغییر داده ایم، دیگر موارد نیز بدون تغییر هستند. در این آزمایش مشاهده کردیم که این مورد باعث کاهش عملکرد مدل از ۵۲٫۷٪ (آزمایش ششم) به ۴۷٫۷٪ شده است.

```
<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=50, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.5, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
  (tanh): Tanh())
```

شکل ۴-۱۳- معماری آزمایش هفتم

```
==== Epoch 1 / 10 ====
Training...
200it [07:27, 2.24s/it]
Batch 200 of 555. Elapsed: 0:07:28.
400it [14:55, 2.24s/it]
Batch 400 of 555. Elapsed: 0:14:56.
555it [20:42, 2.24s/it]

Average training loss generator: 0.537
Average training loss discriminator: 2.057
Training epoch took: 0:20:43

Running Test...
Test Accuracy: 0.477
Test Loss: 1.685
Test took: 0:00:11
```

شکل ۴-۱۴- نتایج آزمایش هفتم

آزمایش هشتم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- اضافه کردن تابع فعالساز Tanh به خروجی مدل G1
- افزایش مقدار احتمال تمام Dropout های استفاده شده در ساختار G1 و D به $p=0.5$ برای کاهش اورفیتینگ مشاهده شده در آزمایشات قبلی
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیمم طول رشته ها : ۲۵۶
- سایز بسته (Batch size) : ۳۲
- دقت تست به دست آمده در ۱۰ ایپاک اول به شرح زیر است :
- ایپاک اول : ۵۶٫۴٪ ایپاک دوم : 60.1٪ ایپاک سوم : ۵۷٫۹٪ ایپاک چهارم : ۵۷٫۳٪ ایپاک پنجم : 59٪

- ایپاک ششم : ۵۹٫۴٪ ایپاک هفتم : ۶۰٫۷٪ ایپاک هشتم : ۵۸٫۵٪ ایپاک نهم : ۵۷٫۹٪ ایپاک دهم : ۵۶٫۸٪

همانطور که مشاهده میشود نسبت به آزمایش شماره پنجم، صرفاً طول رشته را افزایش داده ایم و بجای ۱۲۸ از ۲۵۶ استفاده کرده ایم. که این تغییر موجب بهبود عملکرد مدل از ۵۴٫۶٪ به ۵۶٫۴٪ در ایپاک اول شده است. بهترین عملکرد مدل نیز در ایپاک هفتم میباشد که به دقت تست ۶۰٫۷٪ رسیده است و در مجموع انجام این آزمایش هفت ساعت و نیم به طول انجامید.

```
<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=50, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.5, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
  (tanh): Tanh()
```

شکل ۱۵-۴ - معماری آزمایش هشتم

Average training loss generator: 0.663 Average training loss discriminator: 1.678 Training epoch took: 0:44:49 Running Test... Test Accuracy: 0.564 Test Loss: 1.542 Test took: 0:00:23	Average training loss generator: 0.716 Average training loss discriminator: 1.087 Training epoch took: 0:44:51 Running Test... Test Accuracy: 0.601 Test Loss: 1.534 Test took: 0:00:23	Average training loss generator: 0.714 Average training loss discriminator: 0.957 Training epoch took: 0:44:56 Running Test... Test Accuracy: 0.579 Test Loss: 1.830 Test took: 0:00:23
Average training loss generator: 0.710 Average training loss discriminator: 0.796 Training epoch took: 0:44:57 Running Test... Test Accuracy: 0.585 Test Loss: 1.987 Test took: 0:00:23	Average training loss generator: 0.712 Average training loss discriminator: 0.852 Training epoch took: 0:44:55 Running Test... Test Accuracy: 0.590 Test Loss: 1.978 Test took: 0:00:23	Average training loss generator: 0.711 Average training loss discriminator: 0.821 Training epoch took: 0:44:49 Running Test... Test Accuracy: 0.594 Test Loss: 2.091 Test took: 0:00:23
Average training loss generator: 0.713 Average training loss discriminator: 0.897 Training epoch took: 0:45:00 Running Test... Test Accuracy: 0.573 Test Loss: 2.008 Test took: 0:00:23	Average training loss generator: 0.710 Average training loss discriminator: 0.822 Training epoch took: 0:44:52 Running Test... Test Accuracy: 0.607 Test Loss: 1.677 Test took: 0:00:23	Average training loss generator: 0.709 Average training loss discriminator: 0.793 Training epoch took: 0:44:56 Running Test... Test Accuracy: 0.579 Test Loss: 2.060 Test took: 0:00:23
	Average training loss generator: 0.708 Average training loss discriminator: 0.785 Training epoch took: 0:44:53 Running Test... Test Accuracy: 0.568 Test Loss: 2.256 Test took: 0:00:23	

شکل ۱۶-۴ - نتایج آزمایش هشتم

آزمایش نهم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- اضافه کردن تابع فعالساز Tanh به خروجی مدل G1

- استفاده از GRU در ساختار Discriminator
- افزایش مقدار احتمال تمام Dropout های استفاده شده در ساختار G1 و D به $p=0.5$ برای کاهش اورفیتینگ
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیسم طول رشته ها: ۲۵۶
- سایز بسته (Batch size): ۳۲
- دقت بدست آمده روی داده های تست به شرح زیر است:
- ایپاک اول: ۵۹,۶٪ ایپاک دوم: ۶۱,۳٪ ایپاک سوم: ۶۳,۵٪ ایپاک چهارم: ۶۳,۴٪ ایپاک پنجم: ۶۲,۵٪
- ایپاک ششم: ۵۴,۱٪ ایپاک هفتم: موجود نیست ایپاک هشتم: موجود نیست ایپاک نهم: ۵۸,۳٪ ایپاک دهم: ۵۱,۶٪
- این آزمایش دقیقاً متناظر با آزمایش هشتم است با این تفاوت که به ساختار تمیزدهنده یک GRU اضافه شده است. و در اولین ایپاک از ۵۶,۴٪ (آزمایش هشتم) به ۵۹,۶٪ رسیده است. همانطور که انتظار داشتیم استفاده از ساختار GRU باعث بهبود عملکرد مدل شده است.

```
<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.5, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
  (tanh): Tanh()
)
*****
<bound method Module.parameters of Discriminator(
  (Discriminator_Features): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=768, out_features=512, bias=True)
    (2): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): LeakyReLU(negative_slope=0.2)
    (4): Dropout(p=0.5, inplace=False)
  )
  (GRU): GRU(512, 256, batch_first=True)
)
```

شکل ۴-۱۷- معماری آزمایش نهم

Average training loss generator: 0.753
Average training loss discriminator: 1.055
Training epoch took: 0:45:00

Running Test...
Test Accuracy: 0.613
Test Loss: 1.559
Test took: 0:00:23

Average training loss generator: 0.744
Average training loss discriminator: 0.927
Training epoch took: 0:44:58

Running Test...
Test Accuracy: 0.635
Test Loss: 1.523
Test took: 0:00:23

Average training loss generator: 0.739
Average training loss discriminator: 0.866
Training epoch took: 0:45:00

Running Test...
Test Accuracy: 0.634
Test Loss: 1.702
Test took: 0:00:23

Average training loss generator: 0.763
Average training loss discriminator: 1.541
Training epoch took: 0:44:57

Running Test...
Test Accuracy: 0.596
Test Loss: 1.505
Test took: 0:00:23

```
-----
200it [04:03, 1.22s/it]
Batch 200 of 2,220. Elapsed: 0:04:03.
-----
200it [04:03, 1.22s/it]
Batch 200 of 2,220. Elapsed: 0:04:03.
-----
400it [08:06, 1.22s/it]
Batch 400 of 2,220. Elapsed: 0:08:07.
-----
400it [08:06, 1.22s/it]
Batch 400 of 2,220. Elapsed: 0:08:07.
-----
492it [09:58, 1.22s/it]IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub_msg_rate_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)
1400it [28:24, 1.22s/it]
-----
492it [09:58, 1.22s/it]IOPub message rate exceeded.
The notebook server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--NotebookApp.iopub_msg_rate_limit`.

Current values:
NotebookApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
NotebookApp.rate_limit_window=3.0 (secs)
1400it [28:24, 1.22s/it]
```

Average training loss generator: 0.718 Average training loss discriminator: 0.775 Training epoch took: 0:45:05	Average training loss generator: 0.734 Average training loss discriminator: 0.842 Training epoch took: 0:44:57	Average training loss generator: 0.729 Average training loss discriminator: 0.813 Training epoch took: 0:44:59
Running Test... Test Accuracy: 0.516 Test Loss: 2.663 Test took: 0:00:23	Running Test... Test Accuracy: 0.625 Test Loss: 1.893 Test took: 0:00:23	Running Test... Test Accuracy: 0.541 Test Loss: 2.290 Test took: 0:00:23
Average training loss generator: 0.720 Average training loss discriminator: 0.778 Training epoch took: 0:45:02		
Running Test... Test Accuracy: 0.583 Test Loss: 2.013 Test took: 0:00:23		

شکل ۱۸-۴ - نتایج آزمایش نهم

آزمایش دهم

- ساختار G1 : اضافه کردن یک لایه خطی (FC) با تابع فعالسازی LeakyReLU و اضافه کردن لایه نرمالیزشن (BN layer) در ورودی هر لایه
- استفاده از GRU در ساختار Discriminator
- ۵۰ درصد داده ها برچسب دار هستند
- استفاده از مدل cased برای BERT
- ماکسیمم طول رشته ها : ۲۵۶
- ساینز بسته (Batch size) : ۳۲
- دقت بدست آمده روی داده های تست به شرح زیر است:
- ایپاک اول : ۶۴/۵٪ ایپاک دوم : ۵۹/۷٪ ایپاک سوم : ۶۴/۵٪ ایپاک چهارم : ۵۸٪ ایپاک پنجم : ۶۰/۴٪
- ایپاک ششم : ٪ ایپاک هفتم : ٪ ایپاک هشتم : ایپاک نهم : ایپاک دهم :
- این آزمایش دقیقا مشابه با آزمایش دوم میباشد اما صرفا به ساختار تمیزدهنده یک GRU اضافه شده است.

```
<bound method Module.parameters of Generator(
  (Generator_Network): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): LeakyReLU(negative_slope=0.2)
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Linear(in_features=256, out_features=512, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): Dropout(p=0.1, inplace=False)
    (7): Linear(in_features=512, out_features=768, bias=True)
  )
)
*****
<bound method Module.parameters of Discriminator(
  (Discriminator_Features): Sequential(
    (0): Dropout(p=0.1, inplace=False)
    (1): Linear(in_features=768, out_features=512, bias=True)
    (2): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): LeakyReLU(negative_slope=0.2)
    (4): Dropout(p=0.1, inplace=False)
  )
)
(GRU): GRU(512, 256, batch_first=True)
```

شکل ۱۹-۴ - معماری های آزمایش دهم

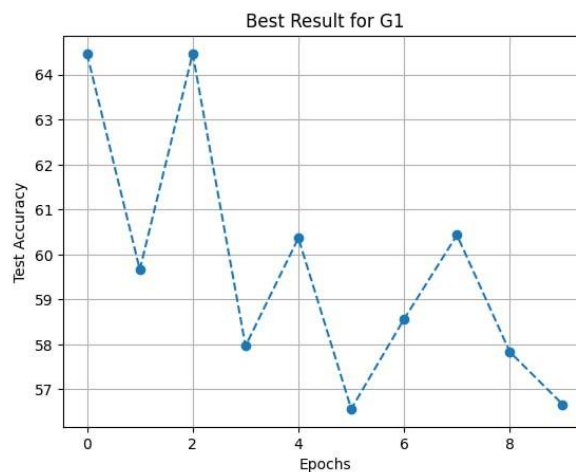
Average training loss generator: 0.727 Average training loss discriminator: 1.359 Training epoch took: 0:45:04 Running Test... Test Accuracy: 0.645 Test Loss: 1.187 Test took: 0:00:23	Average training loss generator: 0.713 Average training loss discriminator: 0.889 Training epoch took: 0:45:00 Running Test... Test Accuracy: 0.645 Test Loss: 1.395 Test took: 0:00:23	Average training loss generator: 0.709 Average training loss discriminator: 0.840 Training epoch took: 0:44:59 Running Test... Test Accuracy: 0.580 Test Loss: 1.980 Test took: 0:00:23
Average training loss generator: 0.707 Average training loss discriminator: 0.811 Training epoch took: 0:44:59 Running Test... Test Accuracy: 0.604 Test Loss: 1.733 Test took: 0:00:23	Average training loss generator: 0.706 Average training loss discriminator: 0.793 Training epoch took: 0:44:59 Running Test... Test Accuracy: 0.566 Test Loss: 2.105 Test took: 0:00:23	Average training loss generator: 0.704 Average training loss discriminator: 0.779 Training epoch took: 0:44:59 Running Test... Test Accuracy: 0.586 Test Loss: 2.017 Test took: 0:00:23
Average training loss generator: 0.703 Average training loss discriminator: 0.770 Training epoch took: 0:44:59 Running Test... Test Accuracy: 0.604 Test Loss: 2.146 Test took: 0:00:23	Average training loss generator: 0.703 Average training loss discriminator: 0.762 Training epoch took: 0:45:00 Running Test... Test Accuracy: 0.578 Test Loss: 2.160 Test took: 0:00:23	Average training loss generator: 0.702 Average training loss discriminator: 0.757 Training epoch took: 0:45:01 Running Test... Test Accuracy: 0.567 Test Loss: 2.384 Test took: 0:00:23

شکل ۴-۲۰- نتایج آزمایش دهم

نتیجه گیری

همانطور که از آزمایش های فوق نتیجه می شود، استفاده از batch normalization و بلوک GRU و همچنین افزایش حداکثر طول توکن ها (متغیر max_len) باعث بهبود عملکرد مدل GAN-BERT می شود.

با درنظر گرفتن نتایج بدست آمده، بهترین نتیجه مربوط به آزمایش دهم بوده و نمودار دقت آن به صورت زیر بر روی داده تست حاصل می شود:



شکل ۴-۲۲- نمودار دقت مدل G_2 روی داده تست با استفاده از ۵۰ درصد داده های برچسب دار برحسب تغییرات اپیک

آزمایشات انجام شده جهت بهبود عملکرد شبکه GAN-BERT با معماری G_2

با توجه به آزمایشات قبلی، در این بخش برای بهبود عملکرد شبکه از بلوک GRU و batch normalization در معماری شبکه discriminator استفاده شده است. همچنین حداکثر طول توکن ها به 256 توکن افزایش یافت. بنابراین کلاس جدید شبکه discriminator به صورت زیر تعریف می شود:

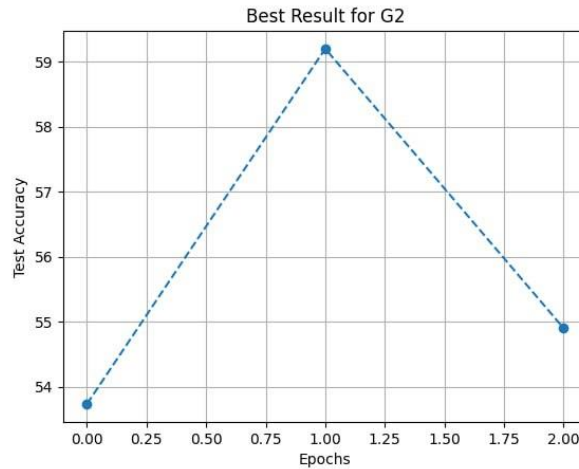
```
class Discriminator(nn.Module):
    def __init__(self, input_dim, hidden_layers, output_layer):
        super(Discriminator, self).__init__()

        self.Discriminator_Features = nn.Sequential(
            nn.Dropout(p=0.1),
            nn.Linear(input_dim, hidden_layers[0]),
            nn.BatchNorm1d(hidden_layers[0]),
            nn.LeakyReLU(0.2),
            nn.Dropout(p=0.1),
        )
        self.GRU = nn.GRU(hidden_layers[0], hidden_layers[1], batch_first=True)
        self.last_linear = nn.Linear(hidden_layers[1], output_layer)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        features = self.Discriminator_Features(x)

        gru_input = features.unsqueeze(1) # Adding time dimension
        gru_output, _ = self.GRU(gru_input)
        gru_output = gru_output[:, -1, :] # Taking the last output
        last_linear_val = self.last_linear(gru_output)
        output = self.softmax(last_linear_val)
        return features, last_linear_val, output
```

سایر مراحل آموزش شبکه و تعریف دیتاست مانند بخش 3 مربوط به استفاده از شبکه G_2 به عنوان Generator است. در این بخش تنها به نتایج شبیه سازی اشاره می شود. با آموزش مدل با استفاده از ۵۰ درصد از داده های لیبل دار، نتیجه دقت مدل در داده تست به صورت زیر حاصل می شود:



شکل ۴-۲۳- نمودار دقت مدل G_2 روی داده تست با استفاده از ۵۰ درصد داده‌های برچسب دار برحسب تغییرات ایپاک

۵. مراجع

- [1] N. Houlsby *et al.*, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019: PMLR, pp. 2790-2799.
- [2] J. Pfeiffer *et al.*, "Adapterhub: A framework for adapting transformers," *arXiv preprint arXiv:2007.07779*, 2020.